Computer Architecture Study
(RISC-V CPU 설계)

김민수

CISC vs RISC

- CISC (Complex Instruction Set Computer)
  - 모든 명령을 갖추고 있는 processer (복잡하고 기능이 많다)
  - 하위 호환성이 좋다
  - 트랜지스터 집적에 있어서 효율성이 떨어진다 (전력 소모 크다, 속도 느림, 가격 비싸다)

- RISC(Reduced Instruction Set Computer)
  - CISC 내부에 갖춘 명령어 중 80%의 일을 처리하는 소수(전체 20%)만 활용
  - 적은 수의 명령어로 구성되어 있어 속도가 빠르고 전력소모가 적다
  - HW가 간단하지만 SW가 크고 복잡하며 하위 호환성이 부족하다

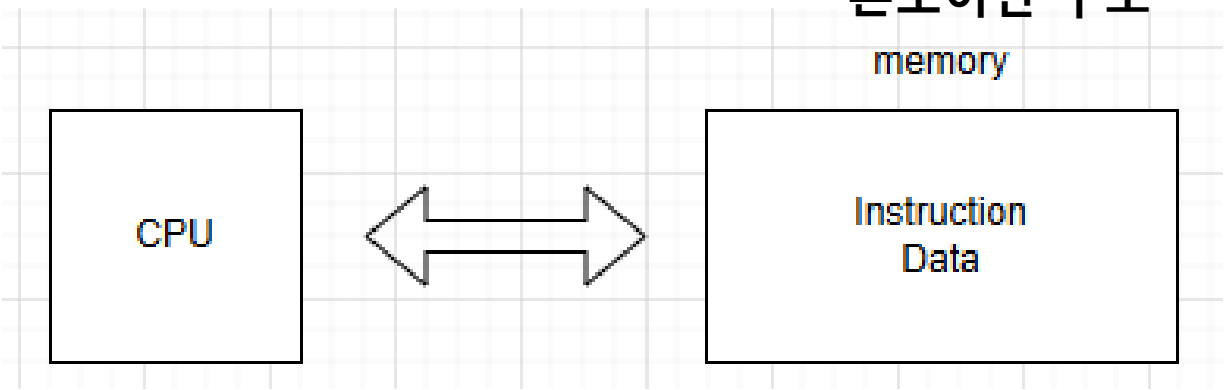
CPU 구조

데이터 메모리와 프로그램 메모리가 구분되어 있지 않음

장점 : 범용성이 크게 향상

단점 : 메모리의 값을 읽고 쓰는 구조이기에 데이터에서 **병목현상**이 생김

폰노이만 구조



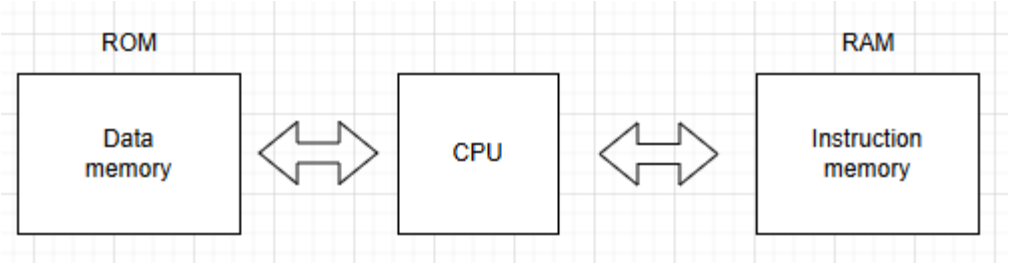
CPU 구조

명령용 버스와 데이터용 버스가
분할되어 있음

장점 : 병목현상이 적어 속도가 빠름

단점 : 복잡하고 전기회로가 많이 필요

하버드 구조



0 + 1 +2 ….. + 9 + 10 까지 sum 하는 CPU 설계

- **C 코드**

```
i = 0
sum = 0

while (i <= 10)
sum = sum + I
out = sum
i = i + 1
holt (종료)
```

이를 바탕으로 설계해보자

각 변수들 선언

1. 데이터 경로 선택 (~srcsel)

- isrcsel : 카운터 레지스터에 넣을 값 선택
- sumsrcsel : 합계 레지스터에 넣을 값 선택
- ALUsrcsel : ALU 두 개의 입력 중 하나에 어떤 값 넣을지 선택하고 ALU 기능 결정

2. 업데이트 신호 (~ load)

- iload : 카운터 레지스터 값을 업데이트 (초기화 or cnt)
- sumload : 합계 레지스터에 값 업데이트 (총합이 올라갈 때)
- outload : 계산이 끝난 후 최종 값을 외부에 보냄

3. 조건 판별

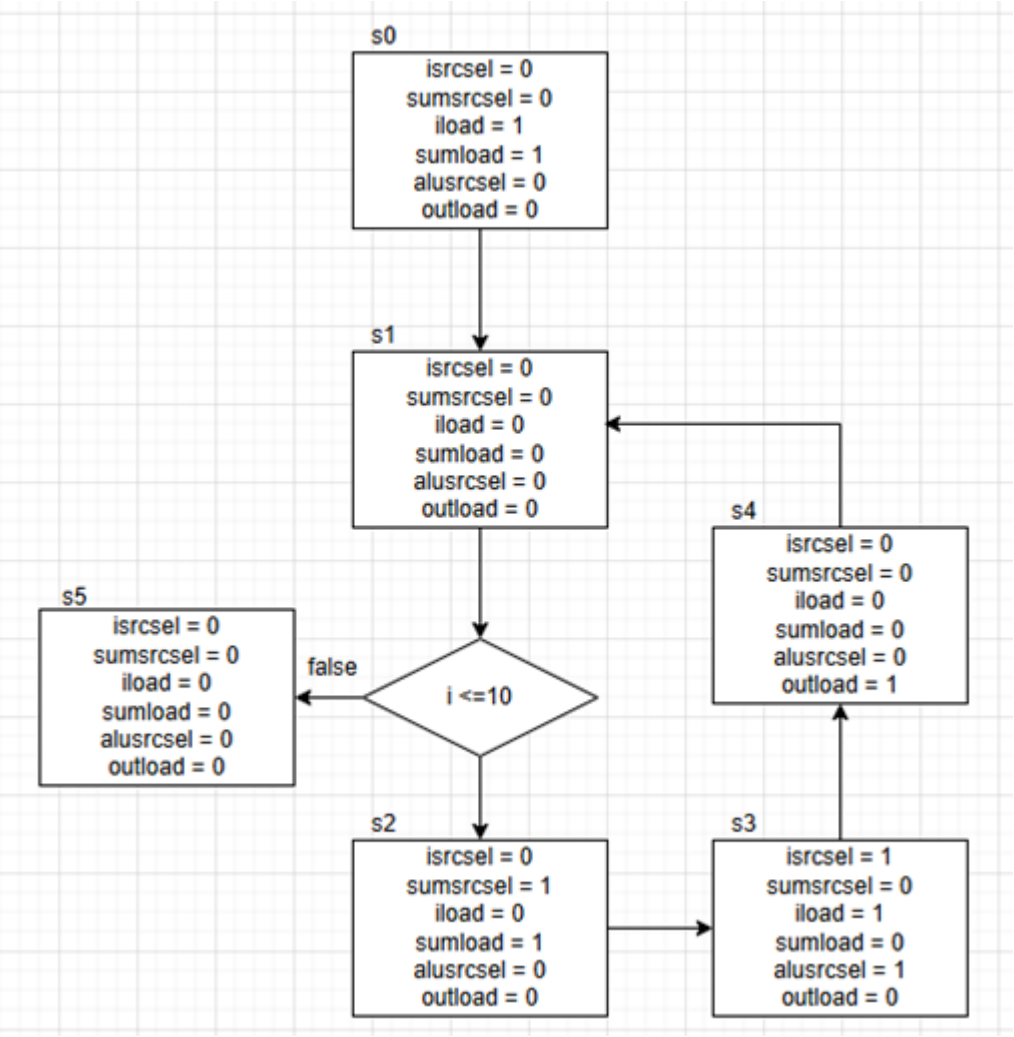
- ilq10 : 10 이하면 계속 루프를 돌고 11이 되는 순간 정지

Truth Table

Instruction	State	isrcsel	sumsrcsel	iload	sumload	ALUsrcsel	outLoad
i = 0, sum = 0	**S0**	0	0	1	1	X	0
ilq10	**S1**	X	X	0	0	X	0
sum = sum + i	**S2**	X	1	0	1	0	0
i = i + 1	**S3**	1	X	1	0	1	0
out = sum	**S4**	X	X	0	0	X	1
halt	**S5**	X	X	0	0	X	0

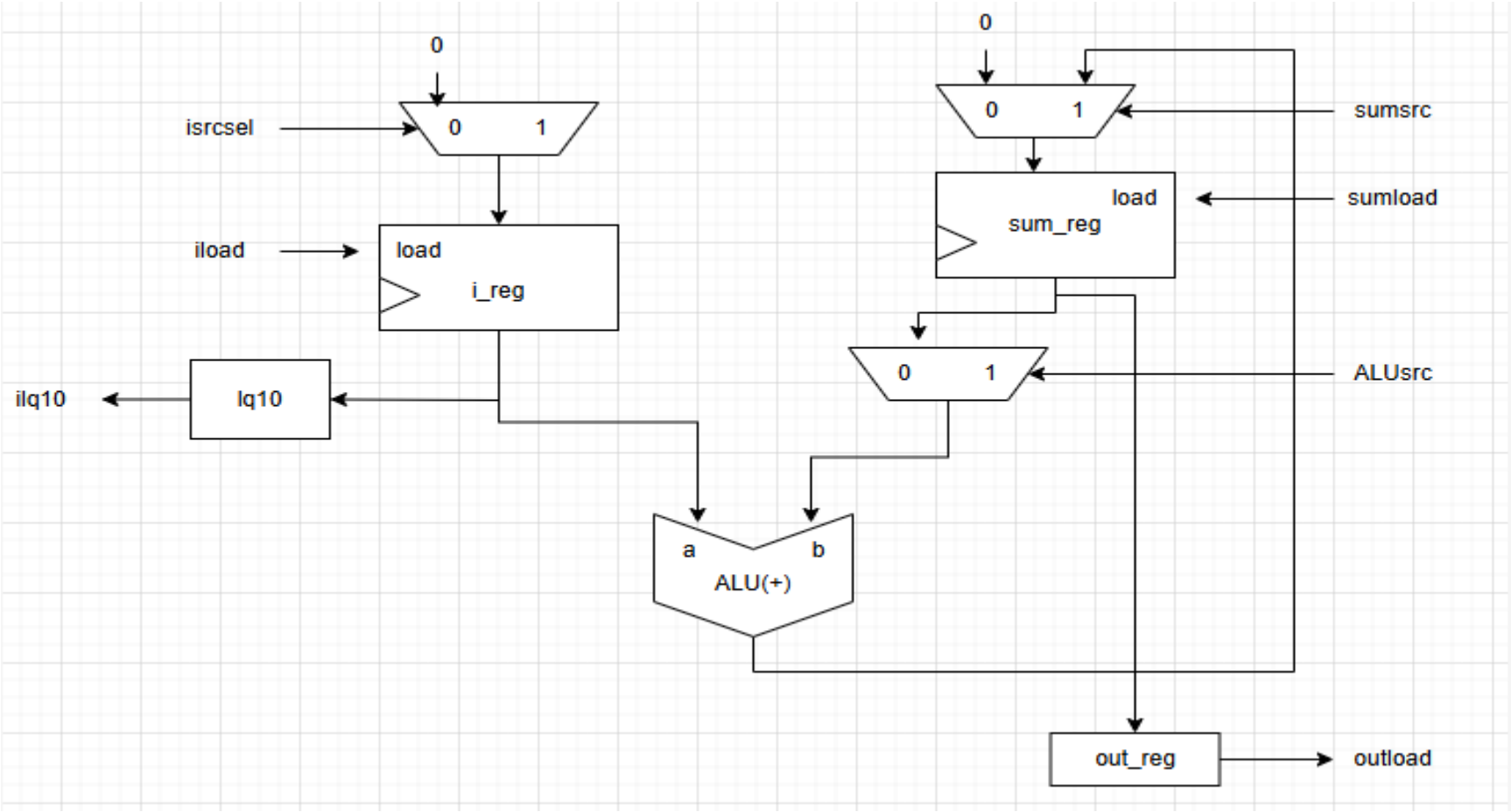
X는 상관 없음을 의미한다

ASM Chart



Truth table과 루프를 반영한 구조도

Block Diagram



Truth table과 ASM을 바탕으로 한 diagram이고 이와 같이 설계하고자 한다

하드웨어 설계(SystemVerilog)

제어 장치 (Control Unit)

```
s2: begin  //합계 누적
    isrcsel = 0;
    sumsrcsel = 1;  // ALU 결과 선택
    iload = 0;
    sumload = 1; // sum register 업데이트
    alusrcsel = 0; // ALU b값 입력
    outload = 0;
    n_state = s3;
end
s3: begin
    isrcsel = 1; // ALU 결과 선택
    sumsrcsel = 0;
    iload = 1;  // i register 업데이트
    sumload = 0;
    alusrcsel = 1;  // ALU b값 입력
    outload = 0;
    n_state = s4;
end
```

Truth table을 바탕으로 설계한 FSM 순차 제어 코드 일부

상태는 총 S0 ~ S5 6가지
조건을 바탕으로 루프를 돌며 최종적으로 S5에서 출력

S2에서는 합계 누적
S3에서는 카운터 증가

하드웨어 설계(SystemVerilog)

Datapath

```
module alu (
    input  [7:0] a,
    input  [7:0] b,
    output [7:0] alu_out
);
    assign alu_out = a + b;
endmodule

module mux_2x1 (
    input  [7:0] a,
    input  [7:0] b,
    input        sel,
    output [7:0] mux_out
);
    assign mux_out = (sel) ? b : a;
endmodule
```

MUX 와 ALU(adder) 의 인스턴스

```
mux_2x1 U_ALU_MUX (
    .a(sumreg_out),
    .b(1),
    .sel(alusrcsel),
    .mux_out(alu_src_data)
);

alu U_ALU (
    .a(ireg_out),
    .b(alu_src_data),
    .alu_out(alu_out)
);
```

하드웨어 설계(SystemVerilog)

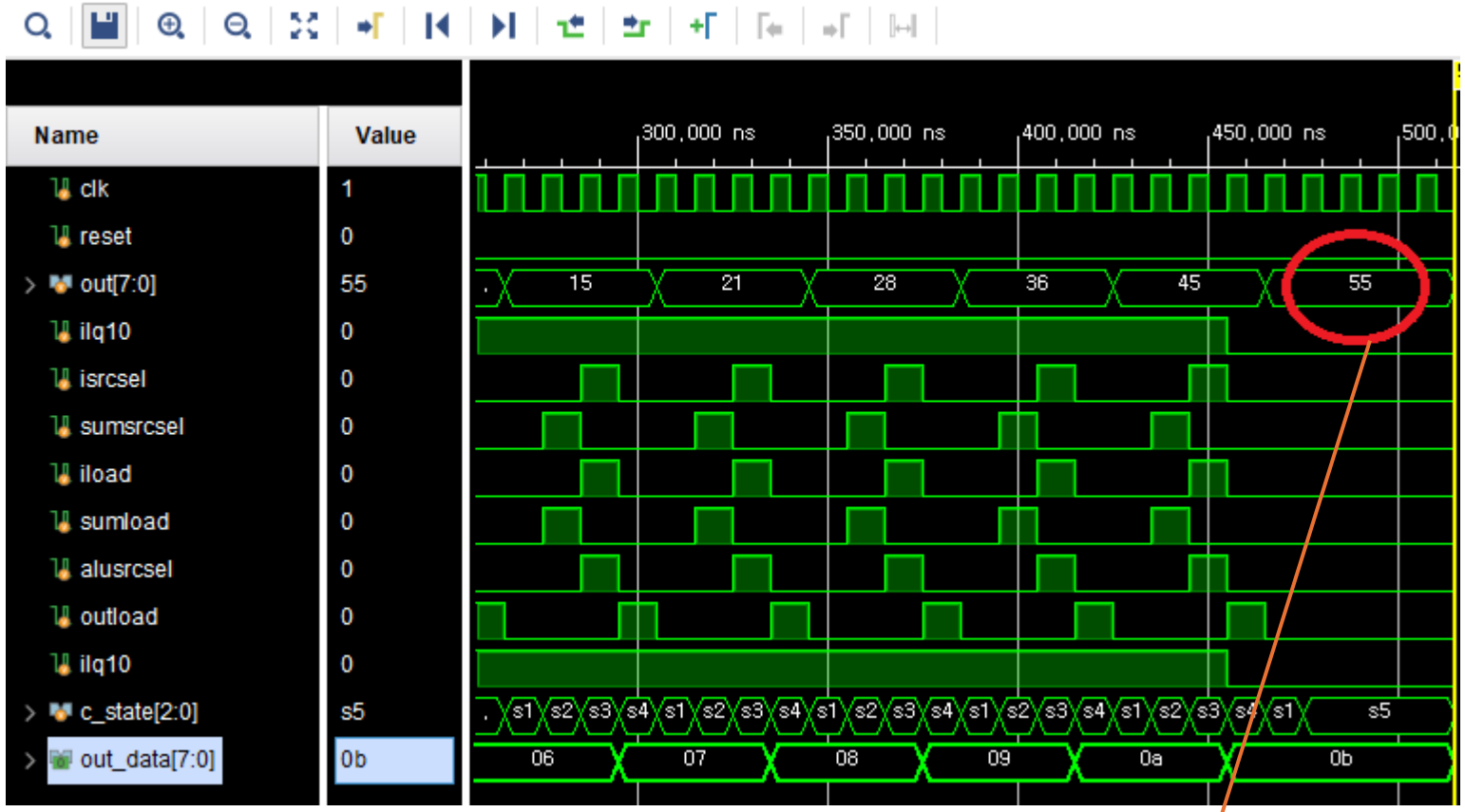
Top Module

```
module dedicated_cpu1 (
    input        clk,
    input        reset,
    output [7:0] out
);
    // 두 모듈을 연결하는 내부 제어 신호 선언
    logic ilq10, isrcsel, sumsrcsel, iload, sumload, alusrcsel, outload;

    control_unit U_CONTROL_UNIT (.*);
    datapath U_DATAPATH (.*);
endmodule
```

Control Unit과 Datapath를 구조화 하여 구성

Simulation

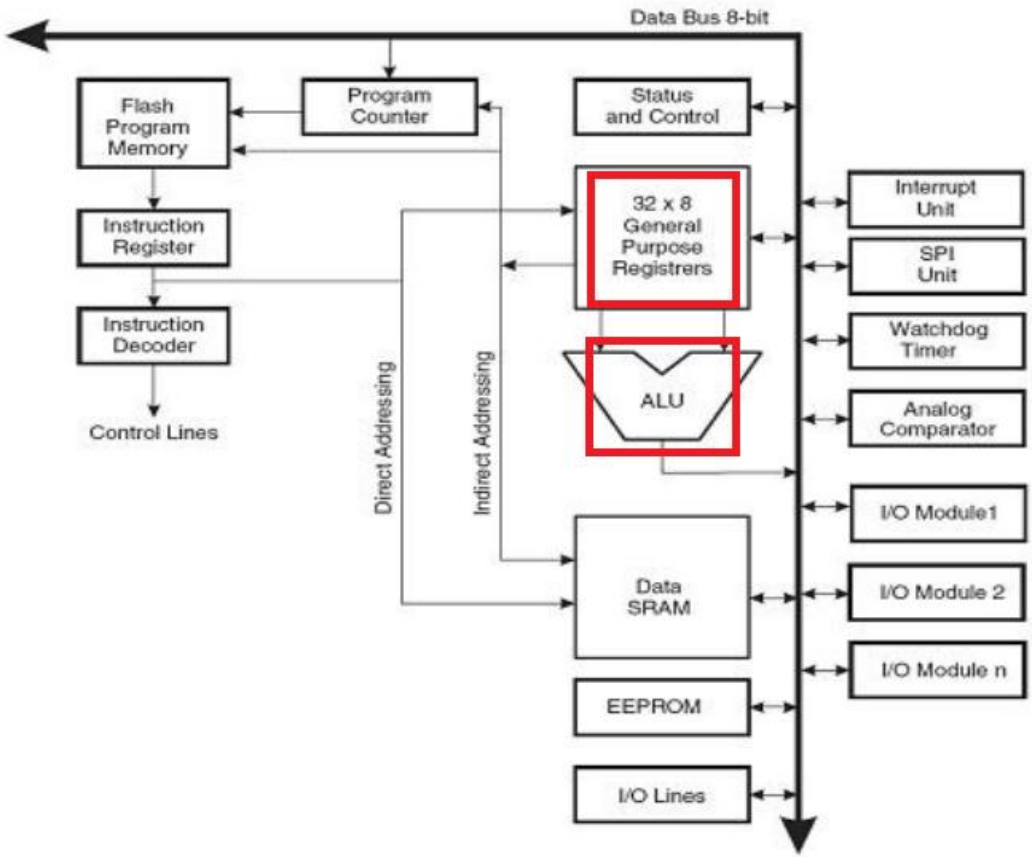
```
logic clk, reset;
logic [7:0] out;

dedicated_cpu0 dut(
    .clk(clk),
    .reset(reset),
    .out(out)
);

always #5 clk = ~clk;
```



clk 설정하고 시간을 보내 결과 확인

최종 계산값

범용 프로세서로 구조 변환



하바드 구조 AVR 아키텍처

Register File : 공용 보관함

ALU : 논리 연산 수행

범용 프로세서로 구조 변환

기존 1~10 가산기를 재설계

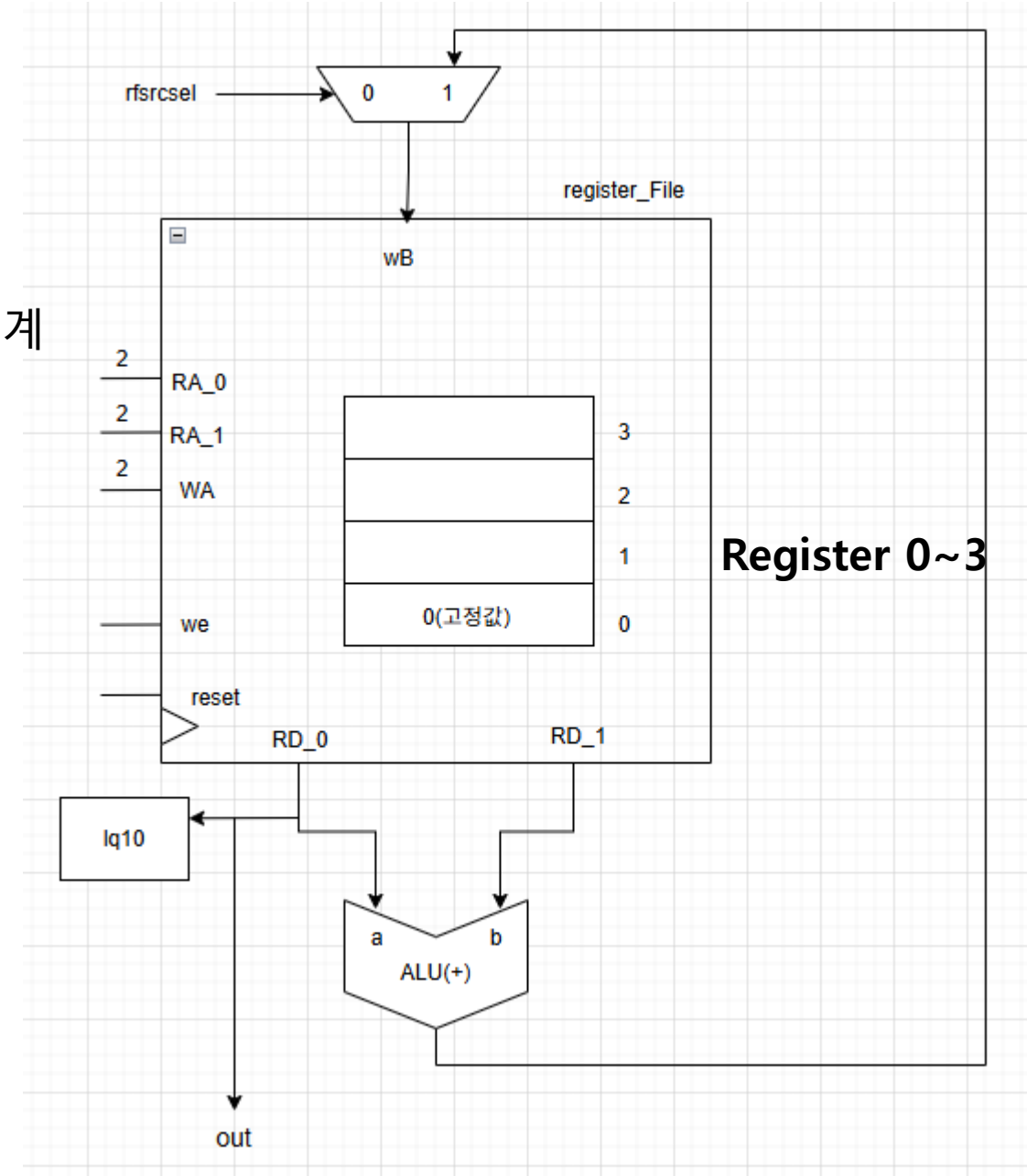
RA_0, RA_1(Read Address) : 데이터를 읽어올 주소
RD_0, RD_1(Read Data) : register에서 읽어온 값

WA(Write Address) : 계산 값을 저장

we(write enable) : 쓰기 허용 신호

wB(write back) : 레지스터로 다시 넣을 데이터 통로

rfsrcsel : 외부 초기값(0) 넣을지 ALU 결과 넣을지 선택



범용 프로세서로 구조 변환

C 언어 형식

R3 = 1

R1 = R0 + R0 (i 초기값)
R2 = R0 + R0 (sum 초기값)

Lq10 ==1

R2 = R2 + R1
R1 = R1 +R3
RD_0 = R2

범용 프로세서로 구조 변환

하드웨어 설계

```
// Register File: R0~R3 공용 레지스터
module register_file (
    input            clk, reset, we,
    input      [1:0] ra0, ra1, wa,
    input      [7:0] wd,
    output     [7:0] rd0, rd1
);
    logic [7:0] reg_mem [3:0];

    // R0는 항상 0으로 고정
    assign rd0 = (ra0 == 0) ? 8'd0 : reg_mem[ra0];
    assign rd1 = (ra1 == 0) ? 8'd0 : reg_mem[ra1];

    always_ff @(posedge clk) begin
        if (reset) begin
            reg_mem[1] <= 0; reg_mem[2] <= 0; reg_mem[3] <= 0;
        end else if (we && wa != 0) begin
            reg_mem[wa] <= wd;
        end
    end
endmodule
```

Register 구성

범용 프로세서로 구조 변환

하드웨어 설계

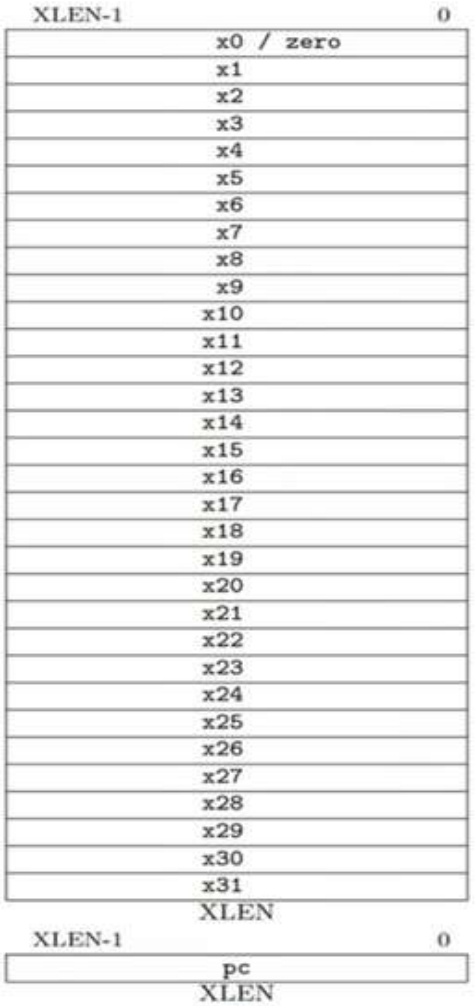
```
always_comb begin
    // 기본값 설정
    n_state  = c_state;
    ra0 = 0; ra1 = 0; wa = 0;
    we = 0; rfsrcsel = 0; outsel = 0;
    case (c_state)
        S_INIT_R3: begin
            wa = 2'd3; we = 1; rfsrcsel = 0; // 외부 입력(1) -> R3 저장
            n_state = S_IDLE;
        end
        S_IDLE: begin // R1=0, R2=0 초기화 (R0+R0 이용)
            ra0 = 2'd0;
            ra1 = 2'd0;
            wa = 2'd1;
            we = 1;
            rfsrcsel = 1; // R1 = 0
            // 실제 구현 시 R2 초기화를 위해 상태를 나누거나 동시 제어 필요
            n_state = S_CHECK;
        end
```

```
S_CHECK: begin
    ra0 = 2'd1; // R1(i) 값을 읽어 비교기로 전달
    if (alt10) n_state = S_SUM;
    else       n_state = S_HALT;
end
S_SUM: begin // R2 = R2 + R1
    ra0 = 2'd2; ra1 = 2'd1; wa = 2'd2; we = 1; rfsrcsel = 1;
    n_state = S_INC;
end
S_INC: begin // R1 = R1 + R3
    ra0 = 2'd1; ra1 = 2'd3; wa = 2'd1; we = 1; rfsrcsel = 1;
    n_state = S_CHECK;
end
S_HALT: begin
    ra0 = 2'd2; outsel = 1; // R2(sum) 값을 출력
    n_state = S_HALT;
end
```

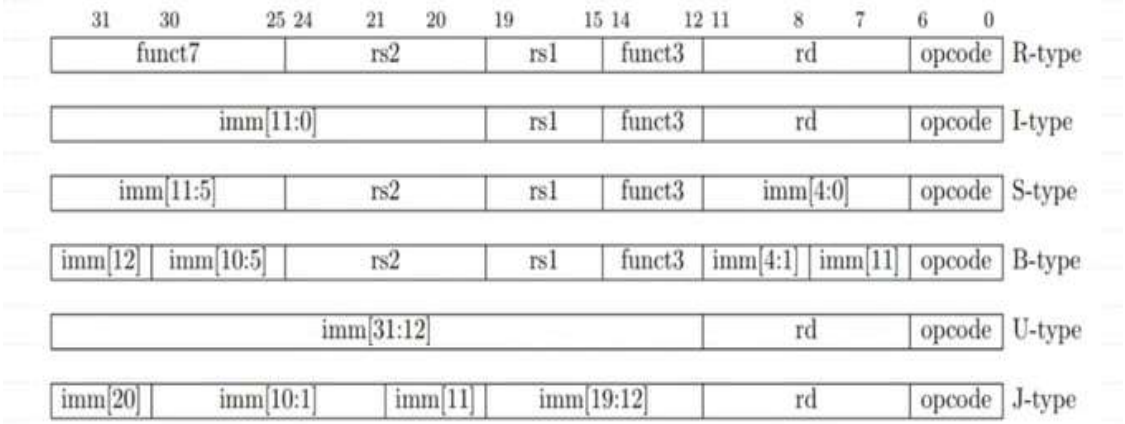
각 상태 정의 후 그에 맞게 구성

RISC-V CPU 설계

앞의 내용들을 바탕으로
Single cycle RV32I를 설계해보자



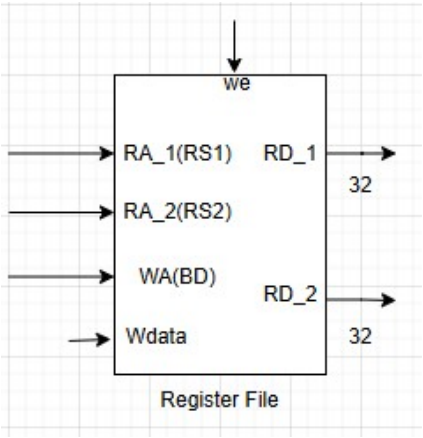
RISC-V CPU 설계

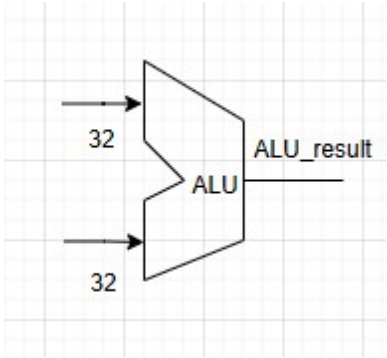


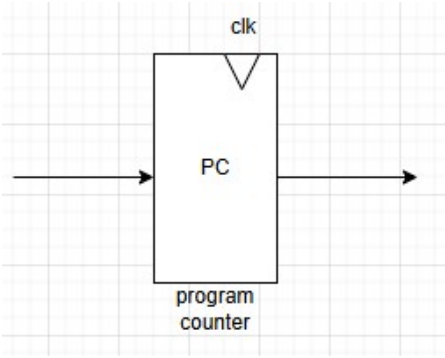
총 6가지의 명령어의 형태가 있는 RV32I

opcode -> 명령어의 타입을 결정

CPU의 기본 모듈





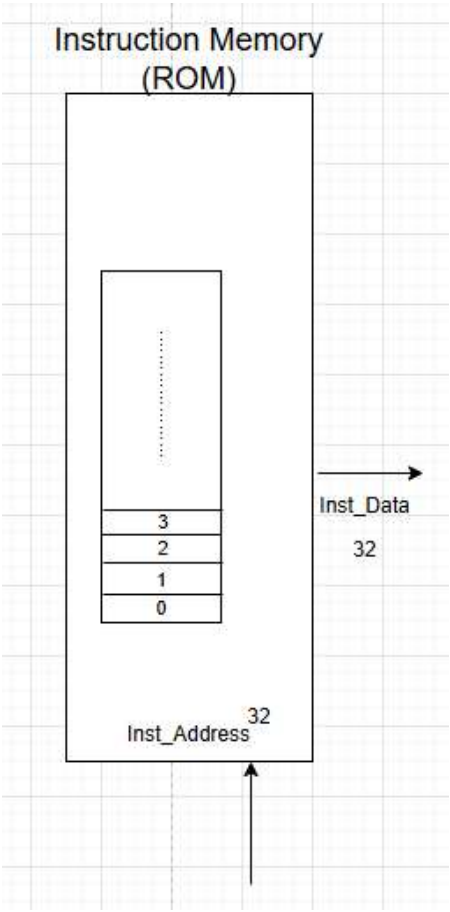


X0~x31 저장하고
읽고 쓰는 모듈

ADD, SUB, AND, OR,
비교 등 연산 수행

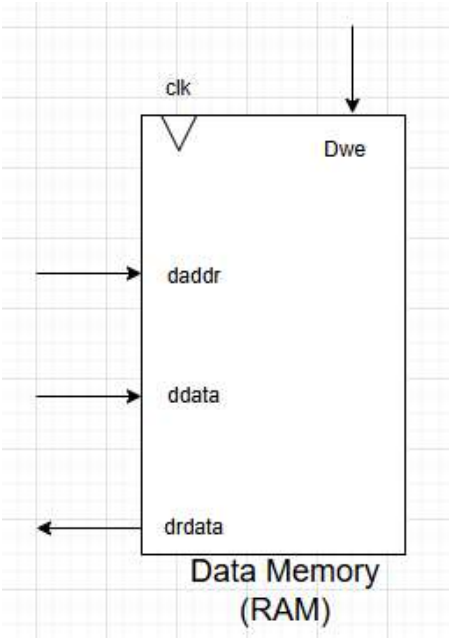
명령어 주소 보관
PC는 +4 증가
(크기 4바이트)

CPU의 기본 모듈



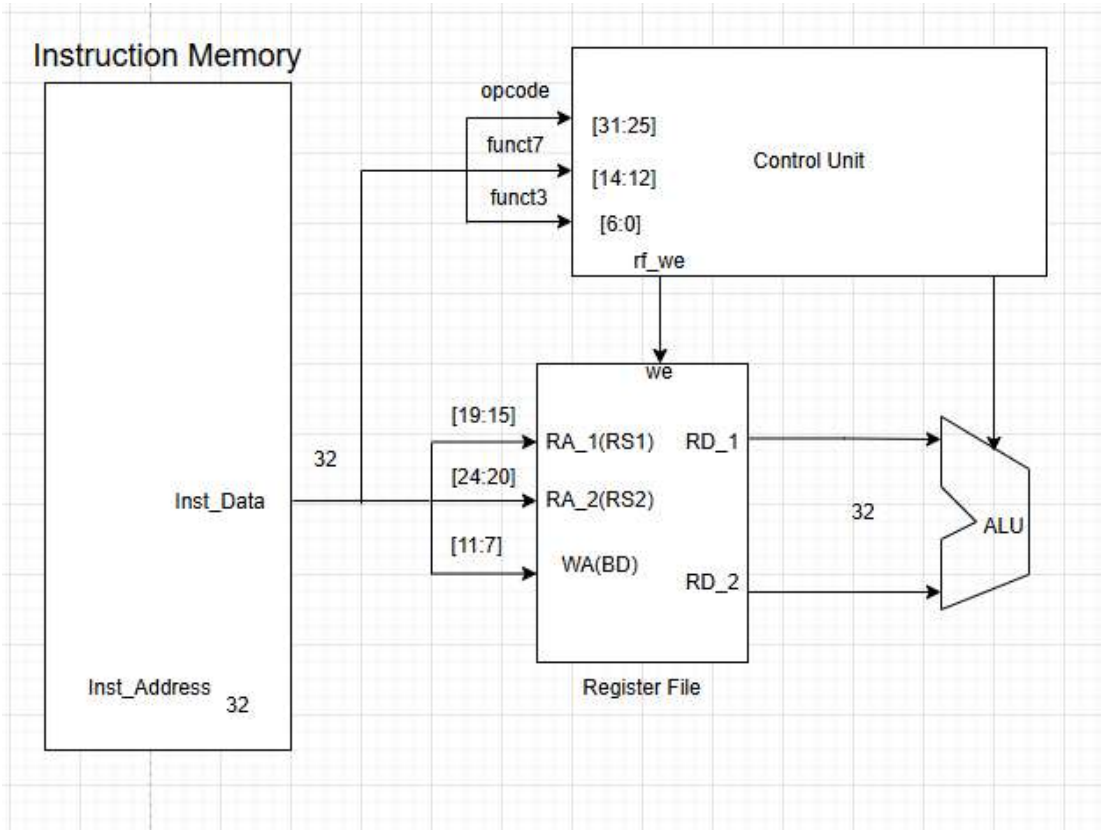
Load/ Store 명령을 위한
메모리

실행할 명령어를 저장



RV32I

31 30 29 28 27 26 25	24 23 22 21 20	19 18 17 16 15	14 13 12	11 10 9 8 7	6 5 4 3 2 1 0	
funct7	rs2	rs1	funct3	rd	opcode	R
imm[12,10:5]	rs2	rs1	funct3	imm[4:1,11]	opcode	B
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	S
imm[11:0]		rs1	funct3	rd	opcode	I
imm[31:12]				rd	opcode	U
imm[20,10:1,11,19:12]				rd	opcode	J



RV32I datasheet를 바탕으로
기본 datapath 구성

Control unit : ROM으로 부터 특정 비트
(opcode, funct7, funct3)를 받아
하드웨어 각 부분에 신호를 전송

RV32I

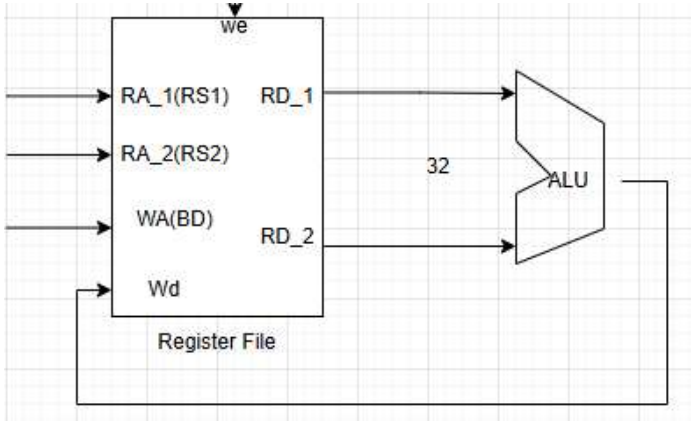
```
module register_file (
    input         clk,
    input         reset,
    input  [ 4:0] RA1,
    input  [ 4:0] RA2,
    input  [ 4:0] WA,
    input  [31:0] Wdata,
    input         rf_we,
    output [31:0] RD1,
    output [31:0] RD2
);
    logic [31:0] register_file[1:31];
```

```
    initial begin
        for (int i = 1; i < 32; i++) begin
            register_file[i] = i;
        end
    end


    always_ff @(posedge clk) begin

        if ((!reset) & rf_we) begin
            register_file[WA] <= Wdata;
        end
    end

    assign RD1 = (RA1 != 0) ? register_file[RA1] : 0;
    assign RD2 = (RA2 != 0) ? register_file[RA2] : 0;
endmodule
```

32비트 데이터를 저장할 수 있는 레지스터 파일

RV32I – R TYPE

0	0	0	0	0	0	0	rs2	rs1	0	0	0	rd	0	1	1	0	0	1	1	R	ADD	rd = rs1 + rs2	
0	1	0	0	0	0	0	rs2	rs1	0	0	0	rd	0	1	1	0	0	1	1	R	SUB	rd = rs1 - rs2	
0	0	0	0	0	0	0	rs2	rs1	0	0	1	rd	0	1	1	0	0	1	1	R	SLL	rd = rs1 << rs2	
0	0	0	0	0	0	0	rs2	rs1	0	1	0	rd	0	1	1	0	0	1	1	R	SLT	rd = (rs1 < rs2)?1:0	
0	0	0	0	0	0	0	rs2	rs1	0	1	1	rd	0	1	1	0	0	1	1	R	SLTU	rd = (rs1 < rs2)?1:0	zero-extends
0	0	0	0	0	0	0	rs2	rs1	1	0	0	rd	0	1	1	0	0	1	1	R	XOR	rd = rs1 ^ rs2	
0	0	0	0	0	0	0	rs2	rs1	1	0	1	rd	0	1	1	0	0	1	1	R	SRL	rd = rs1 >> rs2	
0	1	0	0	0	0	0	rs2	rs1	1	0	1	rd	0	1	1	0	0	1	1	R	SRA	rd = rs1 >> rs2	msb-extends
0	0	0	0	0	0	0	rs2	rs1	1	1	0	rd	0	1	1	0	0	1	1	R	OR	rd = rs1 | rs2	
0	0	0	0	0	0	0	rs2	rs1	1	1	1	rd	0	1	1	0	0	1	1	R	AND	rd = rs1 & rs2	



지정된 두 값(rs1, rs2)를 연산하여 다시 저장

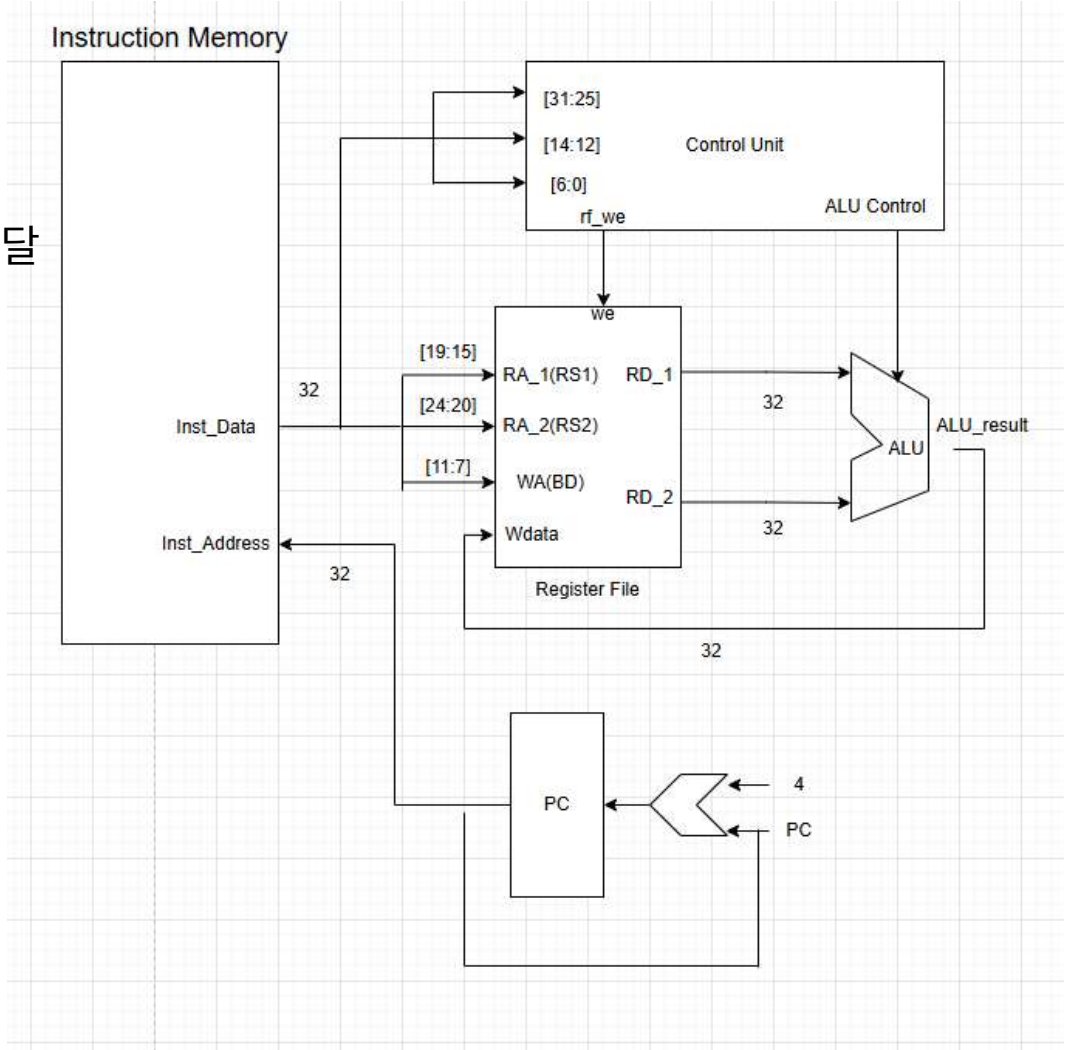
opcode : 0110011
funt7과 funct3의 조합을 이용하여 각 연산 구분

RV32I – R TYPE

PC에 저장된 현재 주소가 Instruction Memory로 전달
메모리에서 해당 주소에 있는 Inst_Data 전송

Control Unit
[6:0] R-Type임을 인식
rfwe 신호를 1로 보내 register로 전송

[31:25] [14:12]를 이용해 연산 종류 지정
ALU_Control 신호 전송



RV32I – R TYPE

```
// R-TYPE Instruction
`define R_TYPE 7'b011_0011
`define ADD 4'b0_000
`define SUB 4'b1_000
`define SLL 4'b0_001
`define SLT 4'b0_010
`define SLTU 4'b0_011
`define XOR 4'b0_100
`define SRL 4'b0_101
`define SRA 4'b1_101
`define OR 4'b0_110
`define AND 4'b0_111
```

```
always_comb begin
    rf_we = 1'b0;
    alu_control = 4'b0000;
    case (opcode)
        `R_TYPE: begin
            rf_we = 1'b1;
            alu_control = {funct7[5], funct3};
        end
```

Header 파일에서
Opcode와 각 연산자
신호 선언

Control unit에서 R-type임을 확인하면
그에 맞는 제어 신호 전송

RV32I – R TYPE

ADD	rd = rs1 + rs2	
SUB	rd = rs1 - rs2	
SLL	rd = rs1 << rs2	
SLT	rd = (rs1 < rs2)?1:0	
SLTU	rd = (rs1 < rs2)?1:0	zero-extends
XOR	rd = rs1 ^ rs2	
SRL	rd = rs1 >> rs2	
SRA	rd = rs1 >> rs2	msb-extends
OR	rd = rs1 | rs2	
AND	rd = rs1 & rs2	

데이터 두 개를 입력 받아
제어 신호(alu_control) 따라
R-Type 명령어 연산 수행

```
module alu (
    input        [31:0] rd1,
    input        [31:0] rd2,
    input        [ 3:0] alu_control,
    output logic [31:0] alu_result,

);
    always_comb begin  //R-TYPE compare
        alu_result = 0;
        case (alu_control)
            `ADD:  alu_result = rd1 + rd2;
            `SUB:  alu_result = rd1 - rd2;
            `SLL:  alu_result = rd1 << rd2[4:0];
            `SLT:  alu_result = ($signed(rd1) < $signed(rd2)) ? 1 : 0;
            `SLTU: alu_result = (rd1 < rd2) ? 1 : 0;
            `XOR:  alu_result = rd1 ^ rd2;
            `SRL:  alu_result = rd1 >> rd2[4:0];
            `SRA:  alu_result = $signed(rd1) >>> rd2;
            `OR:   alu_result = rd1 | rd2;
            `AND:  alu_result = rd1 & rd2;
        endcase
    end
```

SLL/SRL : 시프트는 31번 까지만 의미 있으므로
하위 5비트만 사용

RV32I – R TYPE SIMULATION

							register source 1	register source 2				destination							
0	0	0	0	0	0	0	rs2	rs1	0	0	0	rd	0	1	1	0	0	1	1	R
0	1	0	0	0	0	0	rs2	rs1	0	0	0	rd	0	1	1	0	0	1	1	R
0	0	0	0	0	0	0	rs2	rs1	0	0	1	rd	0	1	1	0	0	1	1	R
0	0	0	0	0	0	0	rs2	rs1	0	1	0	rd	0	1	1	0	0	1	1	R

ex) ADD x5 = x3 + x4 검증

Assembly : 0000000_rs2_rs1_000_rd_0110011

Binary : 0000000_00100_00011_000_00101_0110011
(x4) (x3) (x5) (opcode)

Hex: 004182b3

```
initial begin
    rom[0] = 32'h004182b3; // ADD X5, X3, X4
end
```

RV32I – R TYPE SIMULATION

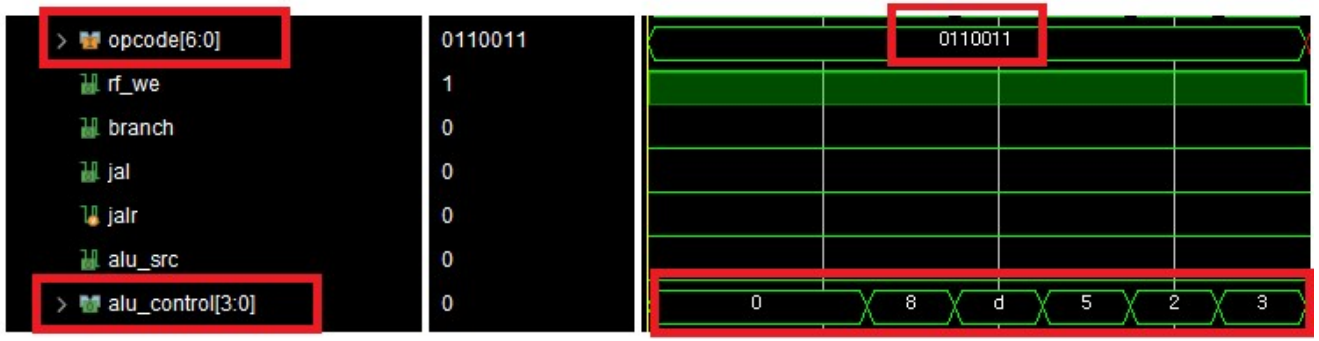
```
initial begin

    rom[0] = 32'h004182b3; // ADD  x5, x3, x4
    rom[1] = 32'h41408533; // SUB  x10, x1, x20
    rom[2] = 32'h402555b3; // SRA  x11, x10, x2
    rom[3] = 32'h00255633; // SRL  x12, x10, x2
    rom[4] = 32'h005526b3; // SLT  x13, x10, x5
    rom[5] = 32'h00553733; // SLTU x14, x10, x5

end
```

ADD : 3 + 4 = 7
SUB : 1- 20 = =19
SRA : x10 오른쪽 두 칸(msb-extend)
SRL : x10 오른쪽 두 칸
SLT : signed 비교
SLTU : unsigned 비교 (zero- extend)

6가지 연산 검증

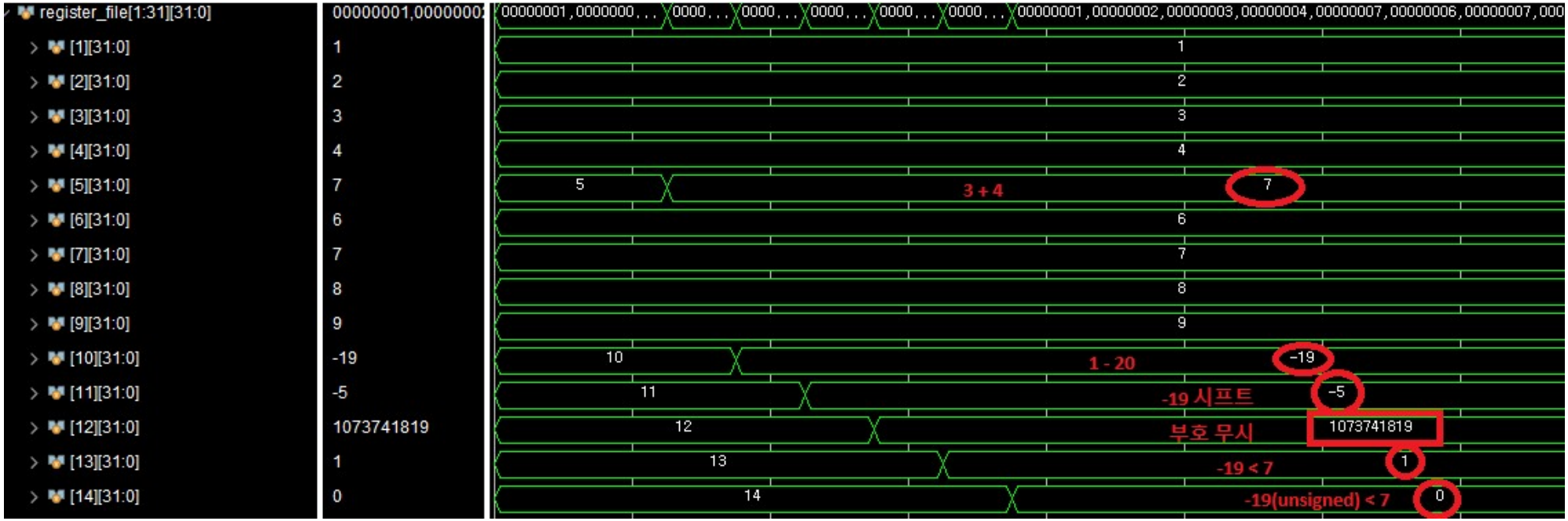
RV32I – R TYPE SIMULATION

R-TYPE



ADD > SUB > SRA > SRL > SLT > SLTU

RV32I – R TYPE SIMULATION



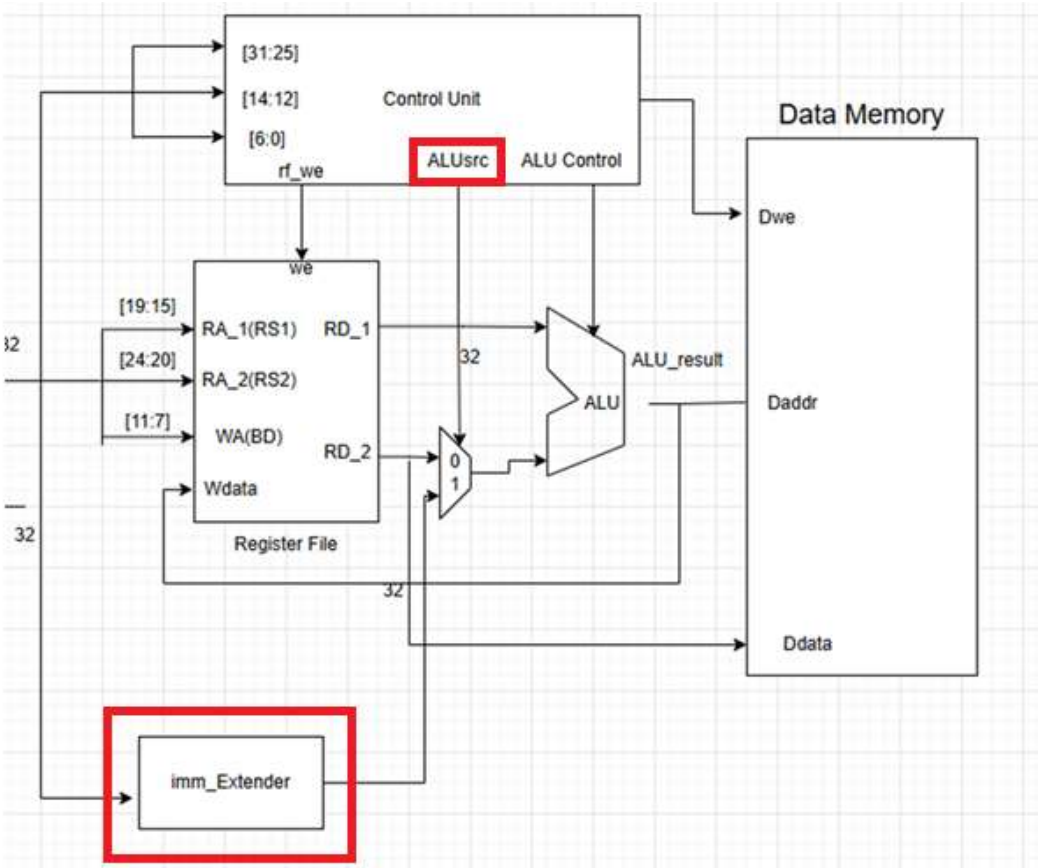
ALU 연산 검증

RV32I – S TYPE

연산의 결과를 memory에 저장

imm_Extender (상수 확장)
 - 명령어에 숫자가 포함되어 있기 때문에 32비트로 길게 늘여주는 역할

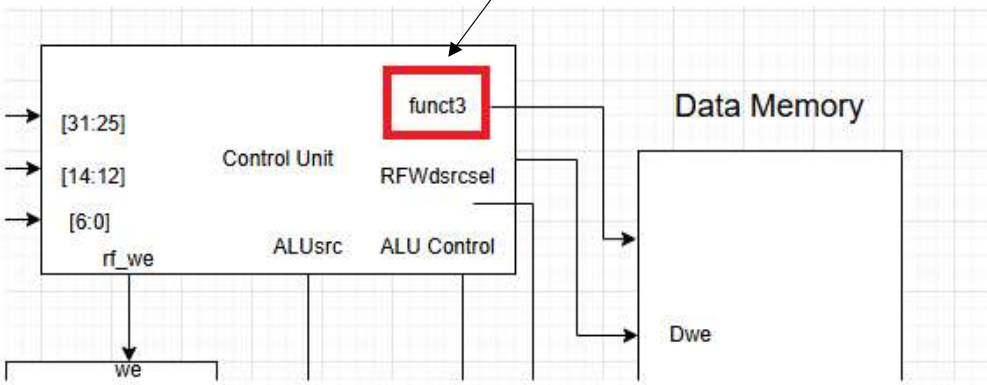
ALUsrc
 - 종류에 따라 상수 값 or RD_2



RV32I – S TYPE

```
//opcode
`define R_TYPE 7'b011_0011
`define S_TYPE 7'b010_0011
```

imm[11:5]	rs2	rs1	0	0	0	imm[4:0]	0	1	0	0	0	1	1	S	SB	M[rs1+imm][0:7] = rs2[0:7]
imm[11:5]	rs2	rs1	0	0	1	imm[4:0]	0	1	0	0	0	1	1	S	SH	M[rs1+imm][0:15] = rs2[0:15]
imm[11:5]	rs2	rs1	0	1	0	imm[4:0]	0	1	0	0	0	1	1	S	SW	M[rs1+imm][0:31] = rs2[0:31]



구분하기 위한 o_funct3 신호 추가

RV32I – S TYPE

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	S

```
module imm_extender (
    input        [31:0] instr_data,
    output logic [31:0] imm_data
);
    always_comb begin
        imm_data = 32'd0;
        case (instr_data[6:0])
            `S_TYPE: begin
                imm_data = {
                    {20{instr_data[31]}}, instr_data[31:25], instr_data[11:7]
                };
            end
```

32비트에 맞게 S TYPE 설정

```
`S_TYPE: begin
    rf_we = 1'b0;
    alu_src = 1'b1;
    alu_control = 4'b000;
    o_funct3 = funct3;
end
```

Control unit에 추가

RV32I – S TYPE

```
always_ff @(posedge clk) begin
    if (dwe) begin
        case (i_funct3)
            3'b010: begin // SW
            dmem[daddr[31:2]] <= dwdata;
            end
            3'b001: begin // SH
            if (daddr[1] == 1'b0) dmem[daddr[31:2]][15:0] <= dwdata[15:0];
            else dmem[daddr[31:2]][31:16] <= dwdata[15:0];
            end
            3'b000: begin // SB
            case (daddr[1:0])
                2'b00: dmem[daddr[31:2]][7:0] <= dwdata[7:0];
                2'b01: dmem[daddr[31:2]][15:8] <= dwdata[7:0];
                2'b10: dmem[daddr[31:2]][23:16] <= dwdata[7:0];
                2'b11: dmem[daddr[31:2]][31:24] <= dwdata[7:0];
            endcase
            end
        endcase
    end
end
```

SW – 전체 데이터 저장

SH – 16비트만 저장
0 > 하위 16비트
1 > 상위 16비트

SB – 4개로 나누어 8비트 저장

RV32I – S TYPE SIMULATION

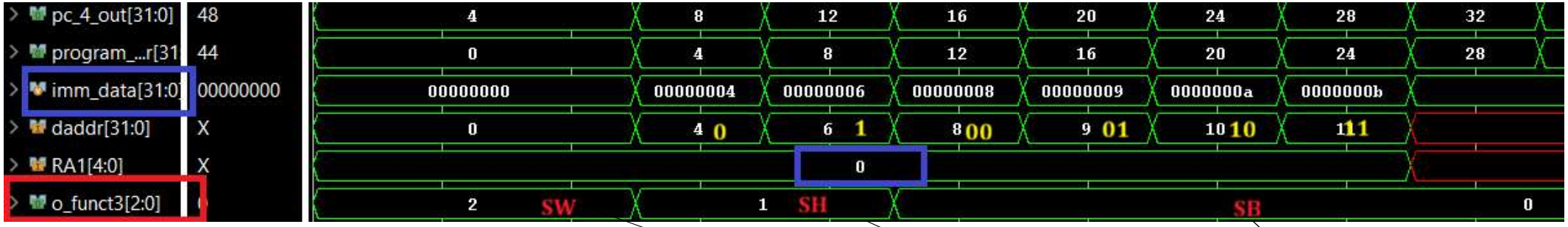
```
logic [31:0] rom[0:127];
  initial begin
      //SW
      rom[0] = 32'h01F02023; // SW x31, 0(x0)

      //SH
      rom[1] = 32'h00101223; // SH x1, 4(x0)
      rom[2] = 32'h00201323; // SH x2, 6(x0)

      // SB
      rom[3] = 32'h00300423; // SB x3, 8(x0)
      rom[4] = 32'h004004A3; // SB x4, 9(x0)
      rom[5] = 32'h00500523; // SB x5, 10(x0)
      rom[6] = 32'h006005A3; // SB x6, 11(x0)
  end
```

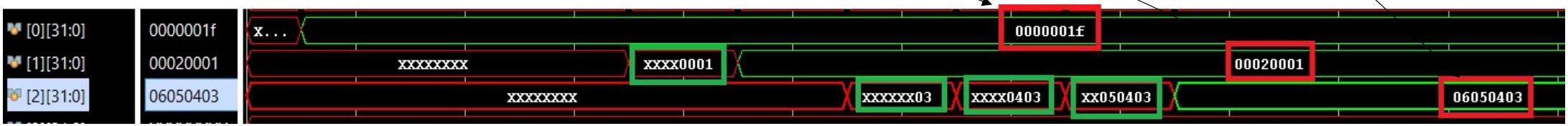
SW 0000001F > dmem[0]

SH 0002, 0001 > dmem[1]

SW 06, 05, 04, 03 > dmem[2]

RV32I – S TYPE SIMULATION





RV32I – I TYPE

메모리에서 불러오기

imm[11:0]								rs1	0	0	0	rd	0	0	0	0	0	1	1	I	LB	rd = M[rs1+imm][0:7]	
imm[11:0]								rs1	0	0	1	rd	0	0	0	0	0	1	1	I	LH	rd = M[rs1+imm][0:15]	
imm[11:0]								rs1	0	1	0	rd	0	0	0	0	0	1	1	I	LW	rd = M[rs1+imm][0:31]	
imm[11:0]								rs1	1	0	0	rd	0	0	0	0	0	1	1	I	LBU	rd = M[rs1+imm][0:7]	zero-extends
imm[11:0]								rs1	1	0	1	rd	0	0	0	0	0	1	1	I	LHU	rd = M[rs1+imm][0:15]	zero-extends
imm[11:0]								rs1	0	0	0	rd	0	0	1	0	0	1	1	I	ADDI	rd = rs1 + imm	
imm[11:0]								rs1	0	1	0	rd	0	0	1	0	0	1	1	I	SLTI	rd = (rs1 < imm)?1:0	
imm[11:0]								rs1	0	1	1	rd	0	0	1	0	0	1	1	I	SLTIU	rd = (rs1 < imm)?1:0	zero-extends
imm[11:0]								rs1	1	0	0	rd	0	0	1	0	0	1	1	I	XORI	rd = rs1 ^ imm	
imm[11:0]								rs1	1	1	0	rd	0	0	1	0	0	1	1	I	ORI	rd = rs1 | imm	
imm[11:0]								rs1	1	1	1	rd	0	0	1	0	0	1	1	I	ANDI	rd = rs1 & imm	
0	0	0	0	0	0	0	shamt	rs1	0	0	1	rd	0	0	1	0	0	1	1	I	SLLI	rd = rs1 << imm[0:4]	
0	0	0	0	0	0	0	shamt	rs1	1	0	1	rd	0	0	1	0	0	1	1	I	SRLI	rd = rs1 >> imm[0:4]	
0	1	0	0	0	0	0	shamt	rs1	1	0	1	rd	0	0	1	0	0	1	1	I	SRAI	rd = rs1 >> imm[0:4]	msb-extends

LB, LH, LW, LBU, LHU -> IL_TYPE
ADDI ~ SRAI -> I_TYPE

```
//opcode
`define R_TYPE 7'b011_0011
`define S_TYPE 7'b010_0011
`define IL_TYPE 7'b000_0011
`define I_TYPE 7'b001_0011
```

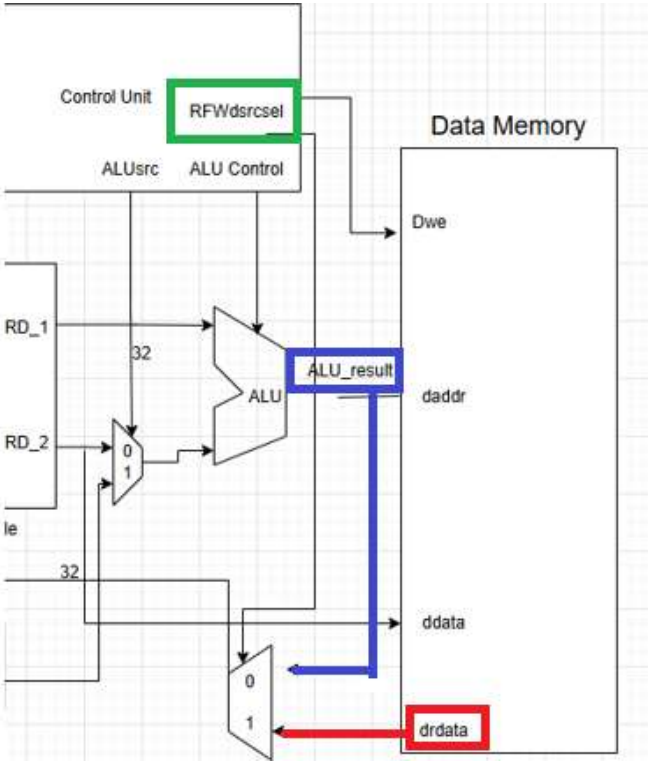
RV32I – I TYPE

RFWdsrcsel

1 : 메모리에서 (drdata)

0 : 연산기에서

```
    mux2X1 U_WRITE_BACK (
        .in0(alu_result),
        .in1(drdata),
        .mux_sel(rfwd_src),
        .out_mux(rf_wb_data)
    );
endmodule
```



RV32I – I TYPE(L)

I	LB	rd = M[rs1+imm][0:7]	
I	LH	rd = M[rs1+imm][0:15]	
I	LW	rd = M[rs1+imm][0:31]	
I	LBU	rd = M[rs1+imm][0:7]	zero-extends
I	LHU	rd = M[rs1+imm][0:15]	zero-extends

funct_3으로 구분

LH, LHU 절반 16비트 가져오기

LB LBU 4등분 8비트 가져오기

Zero-extends – 남은 빈칸 0으로

```
    always_comb begin
        drdata = 32'd0;
        case (i_funct3)
            3'b010: begin  //LW
            drdata = rmem;
            end
            3'b001: begin //LH
            if (daddr[1] == 1'b0) drdata = {{16{rmem[15]}}, rmem[15:0]};
            else drdata = {{16{rmem[31]}}, rmem[31:16]};
            end
            3'b101: begin //LHU
            if (daddr[1] == 1'b0) drdata = {16'd0, rmem[15:0]};
            else drdata = {16'd0, rmem[31:16]};
            end
            3'b000: begin  //LB
            case (daddr[1:0])
                2'b00: drdata = {{24{rmem[7]}}, rmem[7:0]};
                2'b01: drdata = {{24{rmem[15]}}, rmem[15:8]};
                2'b10: drdata = {{24{rmem[23]}}, rmem[23:16]};
                2'b11: drdata = {{24{rmem[31]}}, rmem[31:24]};
            endcase
            end
            3'b100: begin //LBU
            case (daddr[1:0])
                2'b00: drdata = {24'd0, rmem[7:0]};
                2'b01: drdata = {24'd0, rmem[15:8]};
                2'b10: drdata = {24'd0, rmem[23:16]};
                2'b11: drdata = {24'd0, rmem[31:24]};
            endcase
            end
            default: drdata = 0;
        endcase
    end
endmodule
```

RV32I – I TYPE

```
`IL_TYPE: begin
    rf_we = 1'b1;
    alu_src = 1'b1;
    alu_control = 4'b000;
    rfwd_src = 3'b001;
    o_funct3 = funct3;
    dwe = 1'b0;
end
`I_TYPE: begin
    rf_we   = 1'b1;
    alu_src = 1'b1;
    if (funct3 == 3'b001 || funct3 == 3'b101) alu_control = {funct7[5], funct3};
    else alu_control = {1'b0, funct3};
    rfwd_src = 3'b000;
    o_funct3 = funct3;
    dwe = 1'b0;
end
```

Control unit 추가

0	0	0	0	0	0	0	shamt	rs1	0	0	1	rd	0	0	1	0	0	1	1	I	SLLI	rd = rs1 << imm[0:4]
0	0	0	0	0	0	0	shamt	rs1	1	0	1	rd	0	0	1	0	0	1	1	I	SRLI	rd = rs1 >> imm[0:4]
0	1	0	0	0	0	0	shamt	rs1	1	0	1	rd	0	0	1	0	0	1	1	I	SRAI	rd = rs1 >> imm[0:4]

남은 I TYPE은 연산기 결과 그대로 사용

RV32I – I TYPE SIMULATION

```
initial begin
    dmem[2] = 32'ha5010203;
end
```

Memory 초기 값 설정

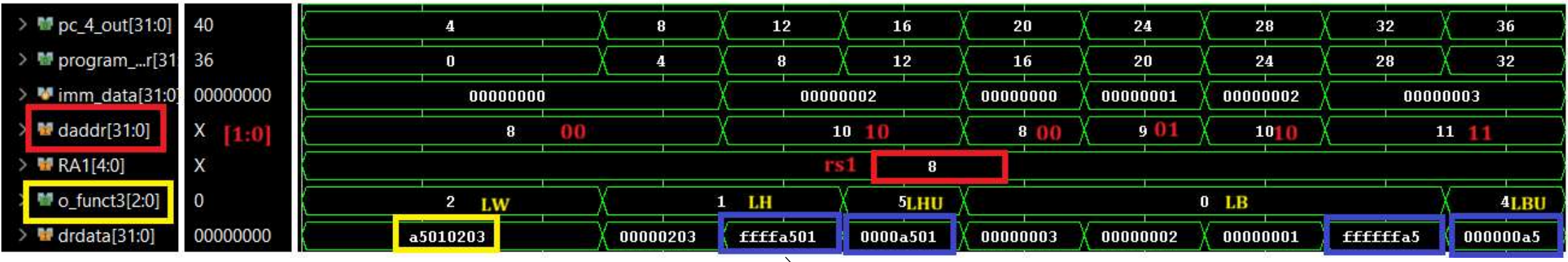
a5, 01, 02, 03

a5 = 10100101 (부호 확장 판단)

```
initial begin
  rom[0] = 32'h00042583;  // LW  x11, 0(x8)

  // --- 2. 16비트 (Halfword) ---
  rom[1] = 32'h00041603;  // LH  x12, 0(x8) -
  rom[2] = 32'h00241683;  // LH  x13, 2(x8)
  rom[3] = 32'h00245703;  // LHU x14, 2(x8)

  // --- 3. 8비트 (Byte) ---
  rom[4] = 32'h00040783;  // LB  x15, 0(x8)
  rom[5] = 32'h00140803;  // LB  x16, 1(x8)
  rom[6] = 32'h00240883;  // LB  x17, 2(x8)
  rom[7] = 32'h00340903;  // LB  x18, 3(x8)
  rom[8] = 32'h00344983;  // LBU x19, 3(x8)
end
```

RV32I – I TYPE SIMULATION

32’ha5010203

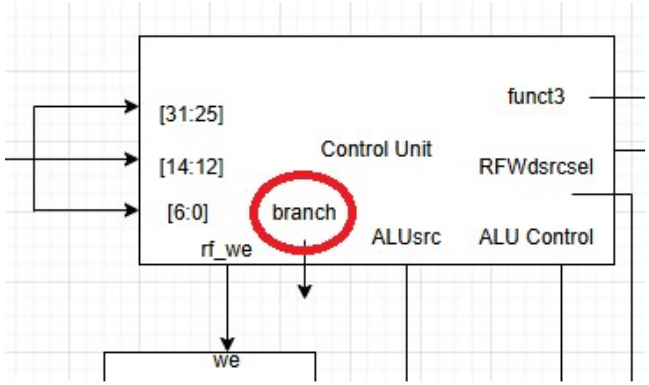


LW > 그대로 32비트 출력
LH > daddr[1] = 0 하위 16비트 , daddr[1] = 1 상위 16비트 (부호 유지)
LHU > 부호 무시 0으로 채움
LB > daddr[1:0] 으로 구분하여 8비트씩 자르기 (부호 유지)
LBU > 부호 무시 0으로 채움

RV32I – B TYPE

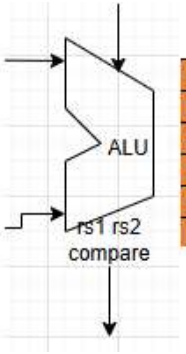
PC로 들어가는 입력 추가

imm[12,10:5]	rs2	rs1	0	0	0	imm[4:1,11]	1	1	0	0	0	1	1	B	BEQ	if(rs1 == rs2) PC += imm	
imm[12,10:5]	rs2	rs1	0	0	1	imm[4:1,11]	1	1	0	0	0	1	1	B	BNE	if(rs1 != rs2) PC += imm	
imm[12,10:5]	rs2	rs1	1	0	0	imm[4:1,11]	1	1	0	0	0	1	1	B	BLT	if(rs1 < rs2) PC += imm	
imm[12,10:5]	rs2	rs1	1	0	1	imm[4:1,11]	1	1	0	0	0	1	1	B	BGE	if(rs1 >= rs2) PC += imm	
imm[12,10:5]	rs2	rs1	1	1	0	imm[4:1,11]	1	1	0	0	0	1	1	B	BLTU	if(rs1 < rs2) PC += imm	zero-extends
imm[12,10:5]	rs2	rs1	1	1	1	imm[4:1,11]	1	1	0	0	0	1	1	B	BGEU	if(rs1 >= rs2) PC += imm	zero-extends



PC를 옮기는 조건 branch 선언

RV32I – B TYPE



B	BEQ	if(rs1 == rs2) PC += imm
B	BNE	if(rs1 != rs2) PC += imm
B	BLT	if(rs1 < rs2) PC += imm
B	BGE	if(rs1 >= rs2) PC += imm
B	BLTU	if(rs1 < rs2) PC += imm
B	BGEU	if(rs1 >= rs2) PC += imm

```
//B-TYPE Instruction
`define BEQ 4'b0_000
`define BNE 4'b0_001
`define BLT 4'b0_100
`define BGE 4'b0_101
`define BLTU 4'b0_110
`define BGEU 4'b0_111
```

비교 연산기 정의 후 설계

btaken

```
    always_comb begin  //B-TYPE compare
        btaken = 0;
        case (alu_control)
            `BEQ: begin
                if (rd1 == rd2) btaken = 1;
                else btaken = 0;
            end
            `BNE: begin
                if (rd1 != rd2) btaken = 1;
                else btaken = 0;
            end
            `BLT: begin
                if ($signed(rd1) < $signed(rd2)) btaken = 1;
                else btaken = 0;
            end
            `BGE: begin
                if ($signed(rd1) >= $signed(rd2)) btaken = 1;
                else btaken = 0;
            end
            `BLTU: begin
                if (rd1 < rd2) btaken = 1;
                else btaken = 0;
            end
            `BGEU: begin
                if (rd1 >= rd2) btaken = 1;
                else btaken = 0;
            end
        endcase
    end
endmodule
```

RV32I – B TYPE

imm[12,10:5]	rs2	rs1	funct3	imm[4:1,11]	opcode	B

```
`B_TYPE: begin
    imm_data = {
        {20{instr_data[31]}},
        instr_data[7],
        instr_data[30:25],
        instr_data[11:8],
        1'b0
    };
end
```

32비트에 맞게 B TYPE 설정

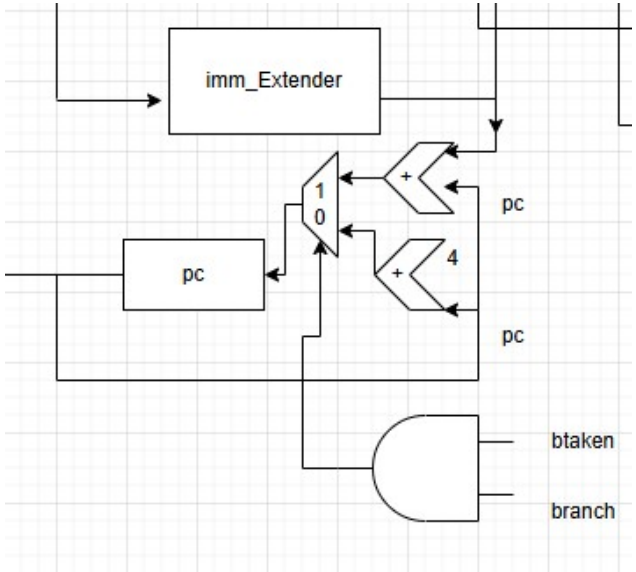
```
`B_TYPE: begin
    rf_we    = 1'b0;
    alu_src = 1'b0;
    alu_control = {1'b0, funct3};
    rfwd_src = 1'b0;
    o_funct3 = 3'b000;
    dwe = 1'b0;
    branch = 1'b1;
end
```

Control unit에 추가

RV32I – B TYPE

branch & btaken 둘 중 하나라도 아니면 0

- 둘다 맞으면 PC + 상수 (imm)
- 0이면 PC+4



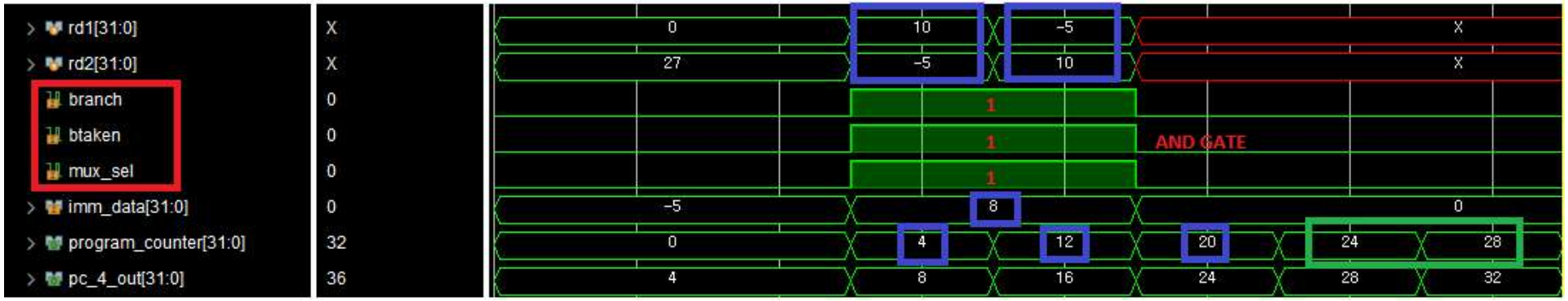
RV32I – B TYPE SIMULATION

```
logic [31:0] rom[0:127];
initial begin
rom[0] = 32'hffb00593; // ADDI x11, x0, -5  (x11에 -5)
rom[1] = 32'h00b51463; // BNE x10, x11, 8
rom[2] = 32'h004182b3; // (건너뛰어야 함) PC + 8
rom[3] = 32'h00a5c463; // BLT x11, x10, 8
rom[4] = 32'h004182b3; // (건너뛰어야 함) PC + 8
end
```

X11에 -5를 저장하여
X10(10) 과 X11(-5) 두 수 비교

BNE -> 10 != -5
BLT -> -5 < 10 (음수 반영)

RV32I – B TYPE SIMULATION



PC 4 >12, 12>20 점프 확인

RV32I – U TYPE

imm[31:12]	rd	0	1	1	0	1	1	1	U	LUI	rd = imm
imm[31:12]	rd	0	0	1	0	1	1	1	U	AUIPC	rd = PC + imm

```
//opcode
`define R_TYPE 7'b011_0011
`define S_TYPE 7'b010_0011
`define IL_TYPE 7'b000_0011
`define I_TYPE 7'b001_0011
`define B_TYPE 7'b110_0011
`define UL_TYPE 7'b011_0111
`define UA_TYPE 7'b001_0111
```

큰 숫자(32비트) 사용

20bit

0 padding

imm[31:12]	rd	opcode	U

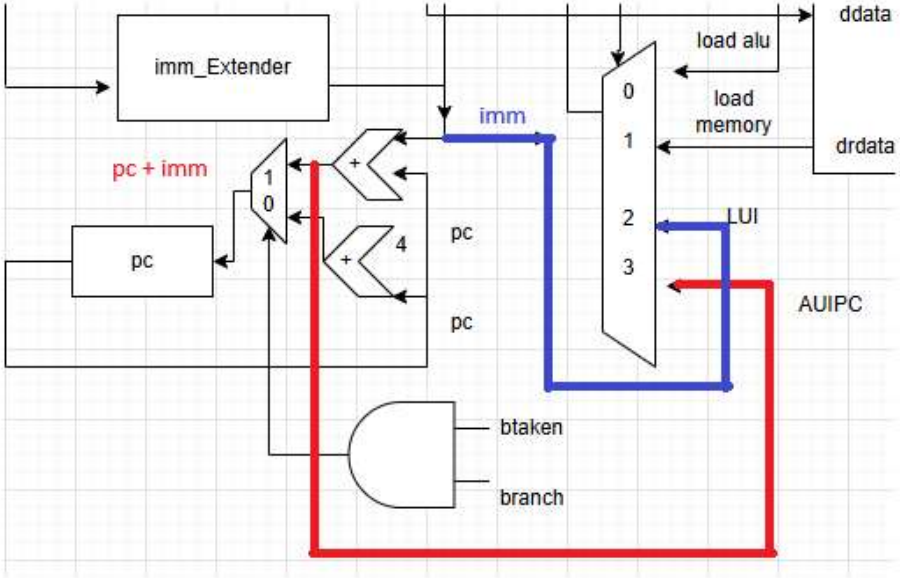
```
`UL_TYPE, `UA_TYPE: begin
    imm_data = {instr_data[31:12], 12'b0000_0000_0000};
end
```

RV32I – U TYPE

```
`UL_TYPE: begin
    rf_we        = 1'b1;
    alu_src      = 1'b0;
    alu_control  = 4'b000;
    rfwd_src     = 3'b010;
    o_funct3     = 3'b000;
    dwe          = 1'b0;
    branch       = 1'b0;
end
`UA_TYPE: begin
    rf_we        = 1'b1;
    alu_src      = 1'b0;
    alu_control  = 4'b000;
    rfwd_src     = 3'b011;
    o_funct3     = 3'b000;
    dwe          = 1'b0;
    branch       = 1'b0;
end
```

쓰기 권한 제외 나머지 기능 0

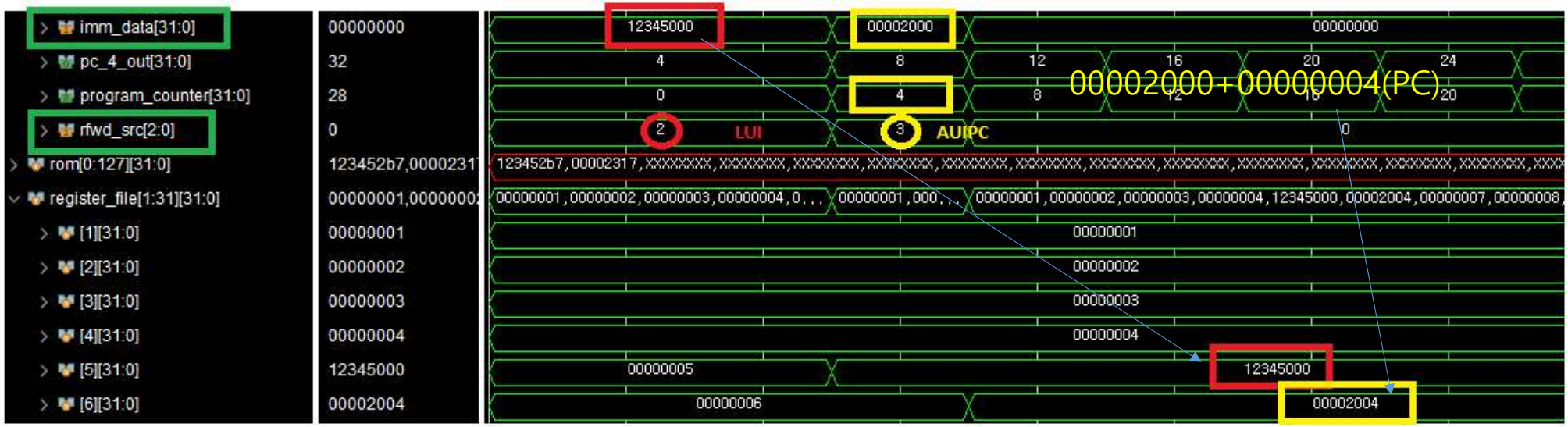
이전 출력 값 그대로 활용



rfwd_src, mux 비트 수정

RV32I – U TYPE SIMULATION

```
logic [31:0] rom[0:127];
initial begin
    rom[0] = 32'h123452b7; // LUI  x5 0x12345
    rom[1] = 32'h00002317; // AUIPC x6 0x00002
end
```



LUI AUIPC 각각 검증

RV32I – J TYPE

imm[11:0]	rs1	0	0	0	rd	1	1	0	0	1	1	1	JL	JALR	rd = PC+4; PC = rs1 + imm
imm[20,10:1,11,19:12]					rd	1	1	0	1	1	1	1	J	JAL	rd = PC+4; PC += imm

```
//opcode
`define R_TYPE 7'b011_0011
`define S_TYPE 7'b010_0011
`define IL_TYPE 7'b000_0011
`define I_TYPE 7'b001_0011
`define B_TYPE 7'b110_0011
`define UL_TYPE 7'b011_0111
`define UA_TYPE 7'b001_0111
`define J_TYPE 7'b110_1111
`define JL_TYPE 7'b110_0111
```

먼 거리 점프

imm[20,10:1,11,19:12]	rd	opcode	J

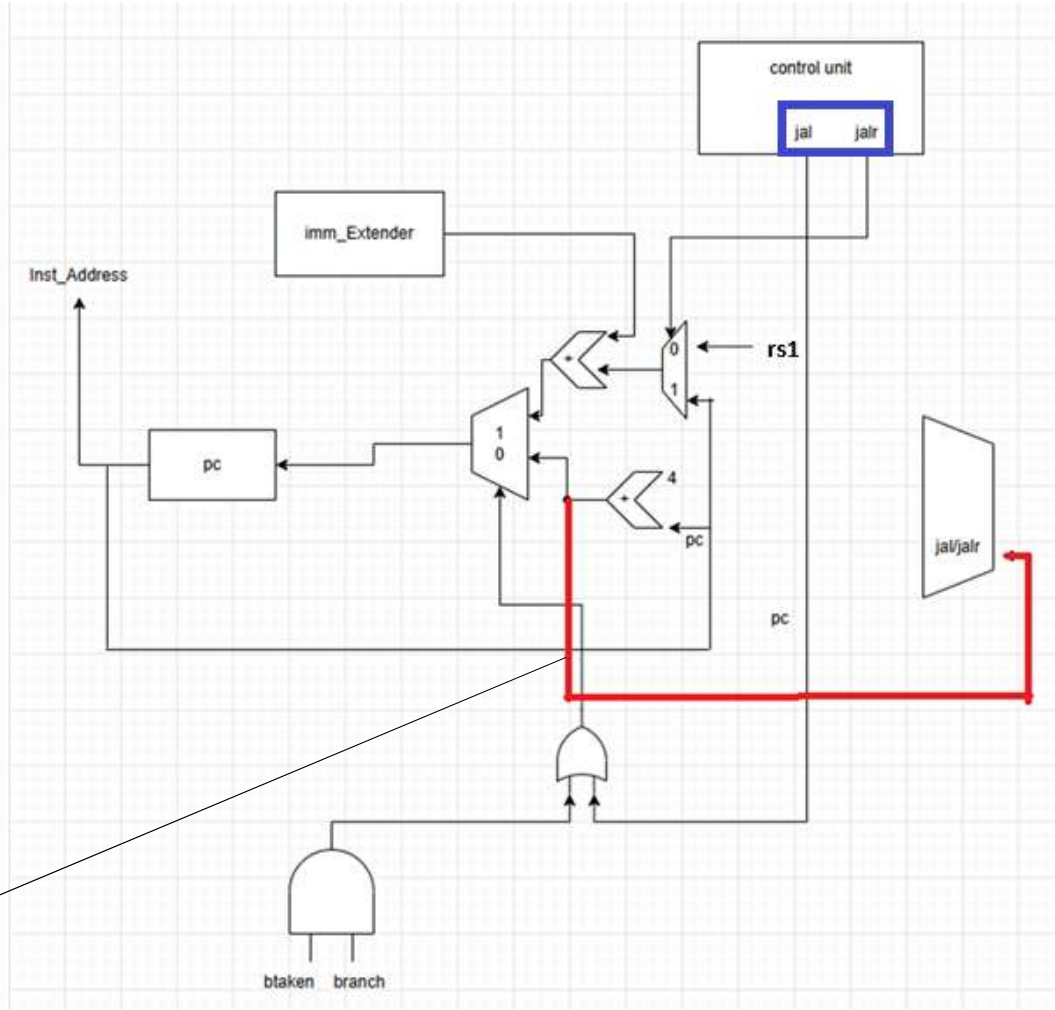
```
`J_TYPE: begin
    imm_data = {
        {12{instr_data[31]}},
        instr_data[19:12],
        instr_data[20],
        instr_data[30:21],
        1'b0
    };
end
```

```
`IL_TYPE, `I_TYPE, `JL_TYPE: begin
    imm_data = {{20{instr_data[31]}}, instr_data[31:20]};
end
```

RV32I – J TYPE

jalr 신호 control unit -> mux sel
0 : pc를 선택
1 : rs1을 선택(rs1 +imm)

Jal 신호 control unit -> OR gate
무조건 점프



rd = PC +4

RV32I – J TYPE

Control unit 추가

```
`J_TYPE, `JL_TYPE: begin
    rf_we       = 1'b1;
    alu_src     = 1'b0;
    alu_control =  4'b000;
    rfwd_src    = 3'b100;
    o_funct3    = 3'b000;
    dwe         = 1'b0;
    branch      = 1'b0;
    jal         = 1'b1;
    if  (opcode == `JL_TYPE) jalr = 1'b1;
    else jalr = 1'b0;
end
```

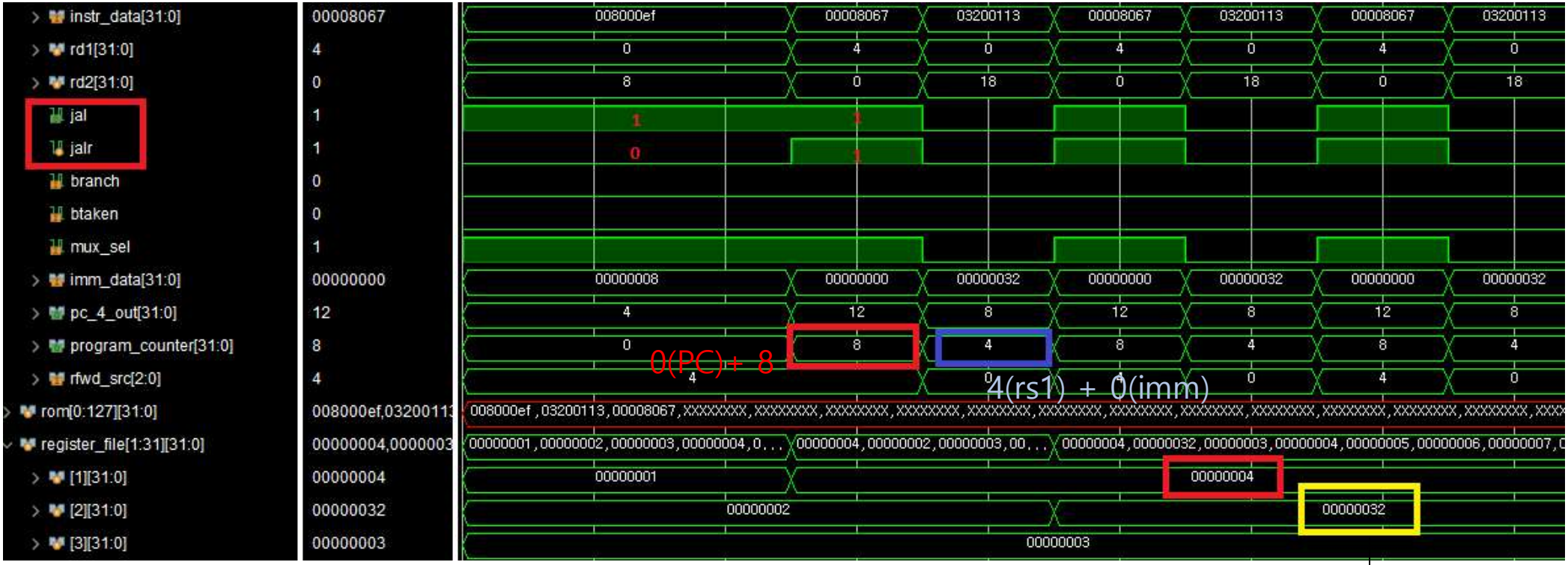
```
module mux5x1 (
    input        [31:0] in0,  // ALU 결과
    input        [31:0] in1,  // drdata
    input        [31:0] in2,  // LUI
    input        [31:0] in3,  // AUIPC
    input        [31:0] in4,  // JALm JALR
    input               mux_sel,
    output logic [31:0] out_mux
);
    always_comb begin
        out_mux = 32'd0;
        case (mux_sel)
            3'b000: out_mux = in0;
            3'b001: out_mux = in1;
            3'b010: out_mux = in2;
            3'b011: out_mux = in3;
            3'b100: out_mux = in4;
        endcase
    end
endmodule
```

rf_we (PC +4 저장)
jal = 1 고정 – 무조건 점프

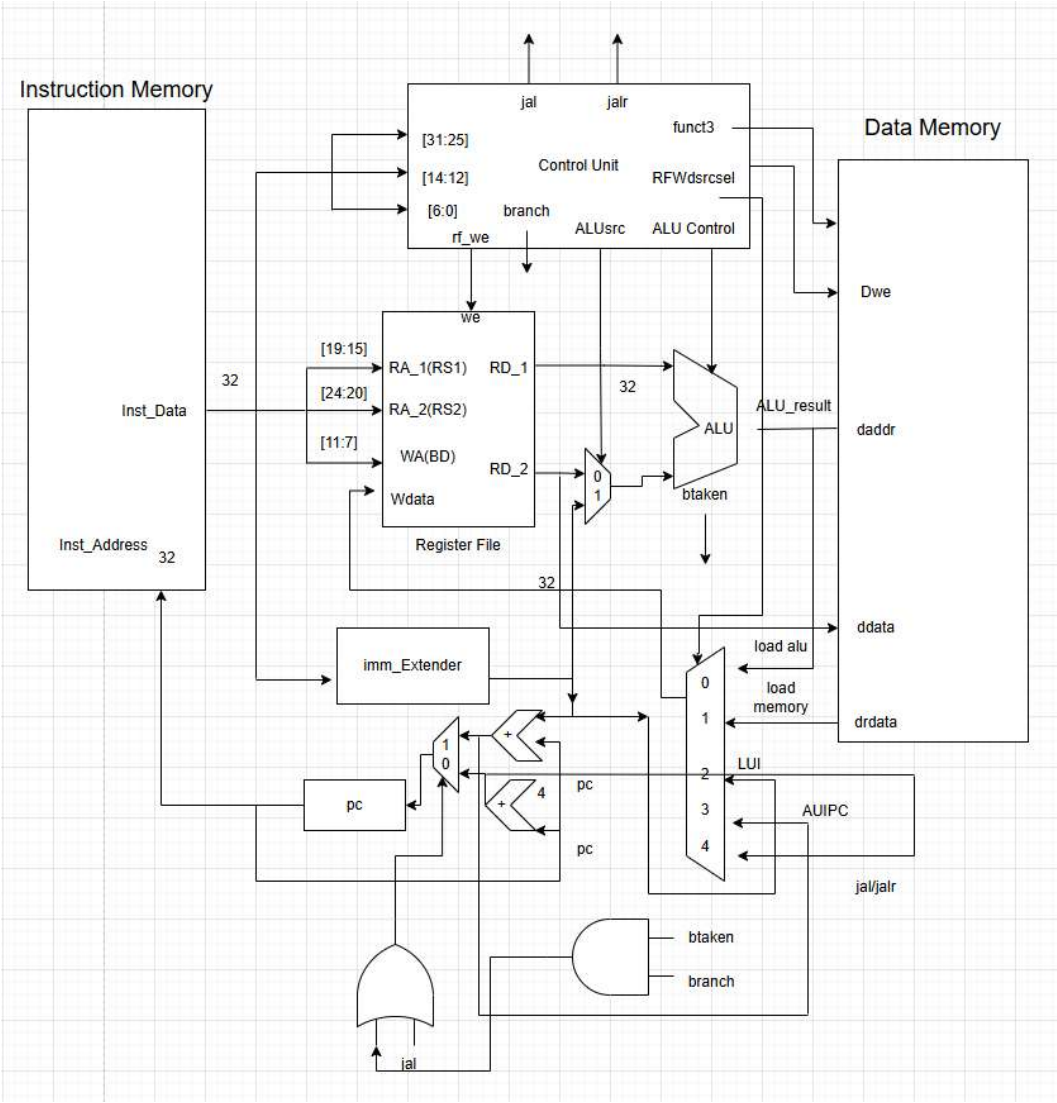
최종 write back MUX

RV32I – J TYPE SIMULATION

```
logic [31:0] rom[0:127];
initial begin
rom[0] = 32'h008000ef; // JAL x1, 8
rom[1] = 32'h03200113; //ADDI x2, x0, 5
rom[2] = 32'h00008067; //JALR x0, 0(x1)
end
```



점프로 인해 건너뛴 명령어 실행



최종 Block Diagram

CPU Compile

0~10 까지 더하는 C 코드

```c
int adder(int a, int b);
void main(void) {
    int i = 0;
    int sum = 0;
    while (i<11) {
        sum = adder(i,sum);
        i = i+1;
    }
    i = 0xfedcba98;
    sum = 0x12345678;
    sum = sum + i;
    while(1);
    return;
}
int adder(int a, int b) {
    return a+b;
}
```

이를 어셈블리어로 변경

CPU Compile

Results

RISC-V (32-bits) gcc

RISC-V (32-bits) gcc (trunk)

```
main:
        addi    sp,sp,-32
        sw      ra,28(sp)
        sw      s0,24(sp)
        addi    s0,sp,32
        sw      zero,-20(s0)
        sw      zero,-24(s0)
        j       .L2
.L3:
        lw      a1,-24(s0)
        lw      a0,-20(s0)
        call    adder
        sw      a0,-24(s0)
        lw      a5,-20(s0)
        addi    a5,a5,1
        sw      a5,-20(s0)
.L2:
        lw      a4,-20(s0)
        li      a5,10
        ble     a4,a5,.L3
        li      a5,-19087360
        addi    a5,a5,-1384
        sw      a5,-20(s0)
        li      a5,305418240
        addi    a5,a5,1656
        sw      a5,-24(s0)
        lw      a4,-24(s0)
        lw      a5,-20(s0)
        add     a5,a4,a5
        sw      a5,-24(s0)
.L4:
        j       .L4
adder:
        addi    sp,sp,-32
        sw      ra,28(sp)
        sw      s0,24(sp)
        addi    s0,sp,32
        sw      a0,-20(s0)
        sw      a1,-24(s0)
        lw      a4,-20(s0)
        lw      a5,-24(s0)
        add     a5,a4,a5
        mv      a0,a5
        lw      ra,28(sp)
        lw      s0,24(sp)
        addi    sp,sp,32
        jr      ra
```

CPU Compile

이를 다시 ROM 코드로 변환

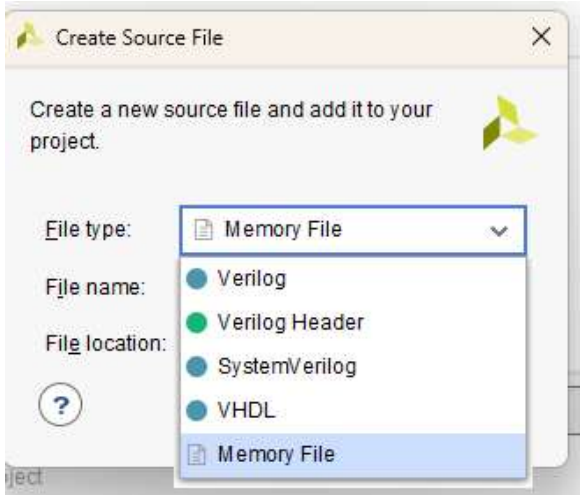
```
.global _boot
.text
li sp, 400
main:
        addi    sp,sp,-32
        sw      ra,28(sp)
        sw      s0,24(sp)
        addi    s0,sp,32
        sw      zero,-20(s0)
        sw      zero,-24(s0)
        j       .L2
.L3:
        lw      a1,-24(s0)
```

```
/* Thanks https://github.com/darklife/darkriscv */
  __heap_size    = 0x200;  /* required amount of heap */
  __stack_size   = 0x400;  /* required amount of stack */

  MEMORY
  {
    ROM (rwx) : ORIGIN = 0x00000000, LENGTH = 0x10000
    RAM (rwx) : ORIGIN = 0x00010000, LENGTH = 0x08000
  }
  SECTIONS
  {
    .text :
    {
```

BUILD

CPU Compile

Memory File 생성해서 붙여 넣기



CPU Compile

risv_rv32.mem

D: > MS > vivado > 202

```
19000113
fe010113
00112e23
00812c23
02010413
fe042623
fe042423
0200006f
fe842583
fec42503
04c000ef
fea42423
fec42783
```

```
initial begin
    $readmemh("risv_rv32.mem",rom);
end
```

불러오기

CPU Compile

while < 11

조건 검사 (L2 점프)

```
        sw      zero,-24(s0)
        j       .L2
.L3:
        lw      a1,-24(s0)
        lw      a0,-20(s0)
        call    adder
        sw      a0,-24(s0)
        lw      a5,-20(s0)
        addi    a5,a5,1
        sw      a5,-20(s0)
.L2:
        lw      a4,-20(s0)
        li      a5,10
        ble     a4,a5,.L3
```

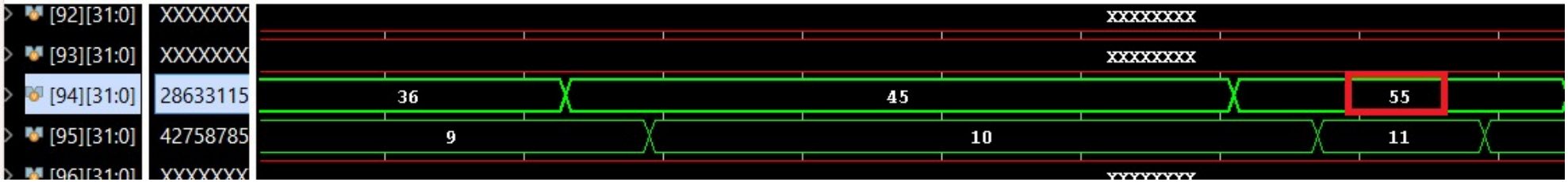
저장된 변수 i 가져옴
비교 대상 10
a4 가 a5 보다 작거나 같으면 L3(루프)

CPU Compile

```
sw      zero,-20(s0)
sw      zero,-24(s0)
```

i = 400 – 20 = 380번지
Sum = 400 -24 = 376 번지

메모리 칸당 4바이트
376 ÷ 4 = 94 (sum)
380 ÷ 4 = 95 (i)

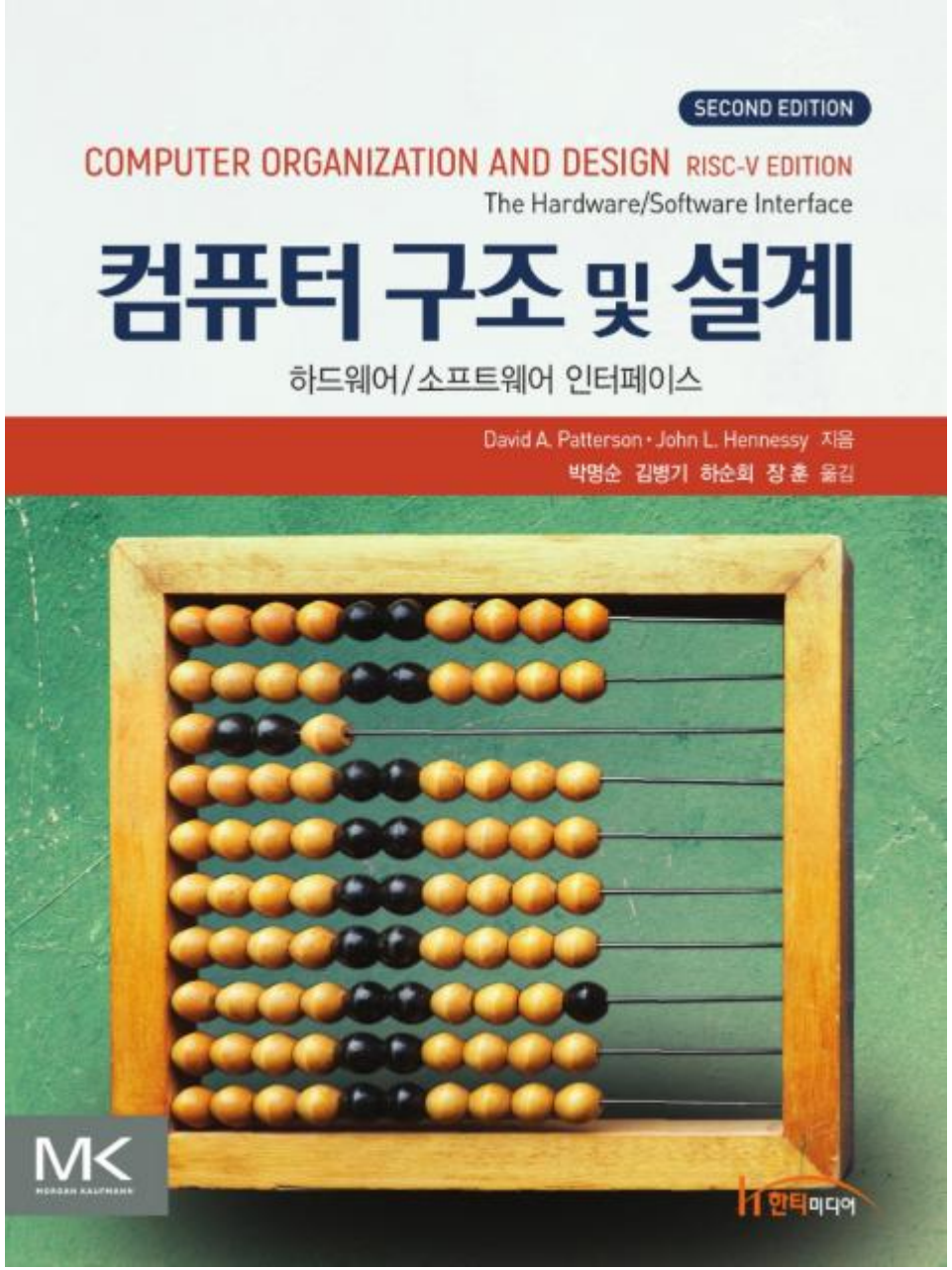


참고 문헌

GitHub - pyong-1459/RISC-V_32I · GitHub

- RISC-V CONTROL.xlsx (RV32I data sheet)

- riscv-spec-20291213.pdf (표준 메뉴얼)

참고 교재

컴퓨터 구조 및 설계 RISC-V 2판